ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA



BÁO CÁO BÀI TẬP LỚN SỐ 2 - PHẦN 1 & 2

MÔN HỆ CƠ SỞ DỮ LIỆU

Học kỳ 261

HỆ THỐNG QUẢN LÝ QUÁN NET

GVHD: **ThS. Trần Thị Quế Nguyệt**

Lớp: L09

Nhóm: 09

Sinh viên thực hiện

Họ và tên	**MSSV**
Nguyễn Thành Đạt	2410709
Nguyễn Minh Đạt	2410694
Nguyễn Phúc Bảo Đạt	2410696
Nguyễn Hữu Tây	2413095
Trần Tiến Thành	2413181

*(*Gạch dưới tên thành viên nộp báo cáo trên LMS)*

Thành phố Hồ Chí Minh, tháng 11 năm 2026

Mục lục

1 Tạo bảng, dữ liệu mẫu và truy vấn dữ liệu **1**

1.1 Các câu lệnh tạo bảng và ràng buộc kiểm tra 1

1.1.1 Tổ chức lược đồ 1

1.1.2 Câu lệnh tạo bảng 1

1.1.3 Khóa ngoại và chính sách toàn vẹn 7

1.2 Câu lệnh tạo chỉ mục 10

1.2.1 Mô tả lý do cần đánh chỉ mục 10

1.2.2 Minh họa hiệu quả truy vấn trước và sau khi đánh chỉ mục 10

1.3 Truy vấn dữ liệu 12

1.3.1 Câu truy vấn 1: Danh sách máy theo khu vực (Q-P12-01) 13

1.3.2 Câu truy vấn 2: Lọc thực đơn theo nhóm và giá (Q-P12-02) 14

1.3.3 Câu truy vấn 3: Tổng hợp đơn đã thanh toán (Q-P12-03) 15

1.3.4 Câu truy vấn 4: Số ca và thời lượng trực (Q-P12-04) 15

1.3.5 Câu truy vấn 5: Hội viên có số dư vượt mức trung bình (Q-P12-05) 17

2 THỦ TỤC – HÀM - TRIGGER **18**

2.1 Thủ tục INSERT / UPDATE / DELETE dữ liệu vào 1 bảng dữ liệu 18

2.1.1 Nguyên tắc xử lý và bảng dữ liệu chính 18

2.1.2 Thủ tục INSERT: Thêm khu vực (sp_khu_vuc_insert) 18

2.1.3 Thủ tục UPDATE: Cập nhật khu vực (sp_khu_vuc_update) 21

2.1.4 Thủ tục DELETE: Xóa khu vực (sp_khu_vuc_delete) 24

2.2 TRIGGER 27

2.2.1 Trigger 1: Bảo vệ biên lai đã thanh toán (T-P12-01) 27

2.2.2 Trigger 2: Đồng bộ tổng tiền đơn từ chi tiết (T-P12-02) 30

2.3 THỦ TỤC 34

2.3.1 Thủ tục 1: Báo cáo doanh thu theo ngày (sp_bao_cao_doanh_thu_ngay) 34

2.3.2 Thủ tục 2: Báo cáo tồn và phân loại dịch vụ (sp_bao_cao_ton_dich_vu) 37

2.4 HÀM 40

2.4.1 Hàm 1: Tổng chi ví ròng (fn_tong_chi_vi_rong) 40

2.4.2 Hàm 2: Tổng tiền từ dòng chi tiết đơn (fn_tong_tien_don) 43

3 PHẦN THÊM: Kiểm chứng và tích hợp ứng dụng **46**

3.1 Kiến trúc và giao dịch 46

3.2 Ba luồng kiểm chứng 46

3.3 Tổng hợp bộ kiểm thử 47
3.3.1 Các ca biên có ảnh hưởng trực tiếp đến tính đúng 47
3.4 Cài mới và đối chiếu từ gói bàn giao 47
3.5 Phạm vi chưa đưa vào demo 47

A BẢNG KÊ NHIỆM VỤ VÀ ĐÁNH GIÁ 48
A.1 Bảng kê nhiệm vụ 48
A.2 Đánh giá của giáo viên 50

B PHỤ LỤC: Thiết kế ERD cập nhật mới nhất 51
B.1 Sơ đồ tổng thể dạng vector 51
B.2 Danh tính và người dùng 52
B.3 Máy trạm và phiên sử dụng 53
B.4 Ví hội viên và chứng từ 54
B.5 Ca trực và phân công 55
B.6 Dịch vụ và đơn hàng 56
B.7 Phiếu kho và kiểm kê 57
B.8 Bảng ghi chú các thay đổi so với BTL1 58

C Dữ liệu mẫu và nguồn minh chứng 59
C.1 Nguồn và khả năng tái tạo 62

PHẦN 1

Tạo bảng, dữ liệu mẫu và truy vấn dữ liệu

1.1 Các câu lệnh tạo bảng và ràng buộc kiểm tra

1.1.1 Tổ chức lược đồ

Hệ thống quản lý đồng thời thời gian sử dụng máy, ví hội viên, bán dịch vụ và tồn kho. Một sự kiện kinh tế được lưu qua chứng từ: phiên và biên lai cho tiền máy; đơn cùng dòng chi tiết cho dịch vụ; phiếu cùng dòng chi tiết cho nhập, xuất hoặc kiểm kê. Cách tổ chức này giúp truy nguyên thay đổi số dư và tồn thay vì chỉ lưu các giá trị tổng hiện tại.

MySQL 8.4.13 sử dụng InnoDB để hỗ trợ khóa ngoại và giao dịch. Các mã định danh dùng so sánh nhị phân `ascii_bin`; dữ liệu văn bản dùng `utf8mb4_0900_as_cs`. Tiền VND được lưu theo số nguyên; giá trị dẫn xuất ở dòng chi tiết được tính từ số lượng và đơn giá.

Bảng 1.1. Năm phân hệ trong lược đồ 23 bảng

Phân hệ	Các bảng	Số bảng
Danh tính	NHOM_NGUOI_DUNG, TAI_KHOAN, SO_DIEN_THOAI, NGUOI_CHOI, NHAN_VIEN, DIA_CHI_NV	6
Máy trạm	KHU_VUC, NHOM_MAY, BANG_GIA, MAY_TRAM, PHIEN_SU_DUNG, BIEN_LAI_TIEN_MAY	6
Dịch vụ	NHOM_DICH_VU, DON_VI_TINH, MAY_IN, DICH_VU, DON_GOI_DICH_VU, CHI_TIET_GOI_DICH_VU	6
Kho	PHIEU_KHO, CHI_TIET_PHIEU_KHO	2
Nhân sự và ví	CA_TRUC, PHAN_CONG_TRUC, GIAO_DICH_VI	3

1.1.2 Câu lệnh tạo bảng

Các tệp sau là nguồn DDL được dùng khi khởi tạo database mới. Khóa chính định danh từng thực thể; UNIQUE ngăn trùng mã nghiệp vụ; CHECK giới hạn trạng thái, miền tiền, số lượng và thứ tự thời gian. Cột sinh kết hợp UNIQUE được dùng cho các trường hợp cần đảm bảo duy nhất có điều kiện, chẳng hạn một máy chỉ có một phiên đang hoạt động.

DDL phân hệ danh tính

```
-- MySQL 8.4.13; baseline 23 tables. Added to unblock backend integration.

CREATE TABLE `NHOM_NGUOI_DUNG` (
  `MaNhomND` VARCHAR(10) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `TenNhomND` VARCHAR(60) NOT NULL,
  PRIMARY KEY (`MaNhomND`),
  CONSTRAINT `uq_nhom_nguoi_dung_0` UNIQUE (`TenNhomND`)
) ENGINE=InnoDB;

CREATE TABLE `TAI_KHOAN` (
  `MaTaiKhoan` VARCHAR(15) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
```

```
  `TenDangNhap` VARCHAR(50) COLLATE utf8mb4_0900_as_ci NOT NULL,
  `MatKhauBam` VARCHAR(255) NOT NULL,
  `Ho` VARCHAR(30) NULL,
  `TenDem` VARCHAR(40) NULL,
  `Ten` VARCHAR(30) NOT NULL,
  `TrangThai` VARCHAR(20) NOT NULL,
  `MaNhomND` VARCHAR(10) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  PRIMARY KEY (`MaTaiKhoan`),
  CONSTRAINT `uq_tai_khoan_0` UNIQUE (`TenDangNhap`),
  CONSTRAINT `ck_tai_khoan_0` CHECK (TrangThai IN ('HoatDong','Khoa','TamDung'))
) ENGINE=InnoDB;

CREATE TABLE `SO_DIEN_THOAI` (
  `MaTaiKhoan` VARCHAR(15) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `SoDienThoai` VARCHAR(20) NOT NULL,
  PRIMARY KEY (`MaTaiKhoan`,`SoDienThoai`)
) ENGINE=InnoDB;

CREATE TABLE `NGUOI_CHOI` (
  `MaTaiKhoan` VARCHAR(15) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `SoCCCD` VARCHAR(20) NOT NULL,
  `SoDuVi` DECIMAL(12,0) NOT NULL,
  `NgayDangKy` DATE NOT NULL,
  PRIMARY KEY (`MaTaiKhoan`),
  CONSTRAINT `uq_nguoi_choi_0` UNIQUE (`SoCCCD`),
  CONSTRAINT `ck_nguoi_choi_0` CHECK (SoDuVi>=0)
) ENGINE=InnoDB;

CREATE TABLE `NHAN_VIEN` (
  `MaTaiKhoan` VARCHAR(15) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `ChucVu` VARCHAR(30) NOT NULL,
  `LuongCoBan` DECIMAL(12,0) NOT NULL,
  `NgayVaoLam` DATE NOT NULL,
  `MaNguoiQuanLy` VARCHAR(15) CHARACTER SET ascii COLLATE ascii_bin NULL,
  PRIMARY KEY (`MaTaiKhoan`),
  CONSTRAINT `ck_nhan_vien_0` CHECK (ChucVu IN ('QuanLy','ThuNgan','PhaChe','KyThuat')),
  CONSTRAINT `ck_nhan_vien_1` CHECK (LuongCoBan>=0)
) ENGINE=InnoDB;

CREATE TABLE `DIA_CHI_NV` (
  `MaTaiKhoan` VARCHAR(15) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `STTDiaChi` INT NOT NULL,
  `SoNha` VARCHAR(50) NULL,
  `Duong` VARCHAR(100) NULL,
  `PhuongXa` VARCHAR(100) NULL,
  `TinhThanh` VARCHAR(100) NOT NULL,
  PRIMARY KEY (`MaTaiKhoan`,`STTDiaChi`),
  CONSTRAINT `ck_dia_chi_nv_0` CHECK (STTDiaChi>0)
) ENGINE=InnoDB;
```

DDL phân hệ máy trạm

```
-- MySQL 8.4.13; baseline 23 tables. Added to unblock backend integration.

CREATE TABLE `KHU_VUC` (
  `MaKhuVuc` VARCHAR(10) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `TenKhuVuc` VARCHAR(60) NOT NULL,
  `MoTaViTri` VARCHAR(255) NULL,
  PRIMARY KEY (`MaKhuVuc`),
```

```
  CONSTRAINT `uq_khu_vuc_0` UNIQUE (`TenKhuVuc`)
) ENGINE=InnoDB;

CREATE TABLE `NHOM_MAY` (
  `MaNhomMay` VARCHAR(10) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `TenNhomMay` VARCHAR(60) NOT NULL,
  `DonGiaGioMacDinh` DECIMAL(10,0) NOT NULL,
  `MoTa` VARCHAR(255) NULL,
  PRIMARY KEY (`MaNhomMay`),
  CONSTRAINT `uq_nhom_may_0` UNIQUE (`TenNhomMay`),
  CONSTRAINT `ck_nhom_may_0` CHECK (DonGiaGioMacDinh>=0)
) ENGINE=InnoDB;

CREATE TABLE `BANG_GIA` (
  `MaNhomND` VARCHAR(10) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `MaNhomMay` VARCHAR(10) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `DonGiaGio` DECIMAL(10,0) NOT NULL,
  PRIMARY KEY (`MaNhomND`,`MaNhomMay`),
  CONSTRAINT `ck_bang_gia_0` CHECK (DonGiaGio>=0)
) ENGINE=InnoDB;

CREATE TABLE `MAY_TRAM` (
  `MaMay` VARCHAR(10) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `TenMay` VARCHAR(50) NOT NULL,
  `TrangThaiMay` VARCHAR(20) NOT NULL,
  `ViTriHang` INT NULL,
  `ViTriCot` INT NULL,
  `MaNhomMay` VARCHAR(10) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `MaKhuVuc` VARCHAR(10) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  PRIMARY KEY (`MaMay`),
  CONSTRAINT `uq_may_tram_0` UNIQUE (`TenMay`),
  CONSTRAINT `ck_may_tram_0` CHECK (TrangThaiMay IN
  ↪  ('AVAILABLE','ONLINE','MAINTENANCE','OFFLINE'))
) ENGINE=InnoDB;

CREATE TABLE `PHIEN_SU_DUNG` (
  `MaPhien` VARCHAR(20) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `HinhThucDangNhap` VARCHAR(20) NOT NULL,
  `ThoiDiemBatDau` DATETIME NOT NULL,
  `ThoiDiemKetThuc` DATETIME NULL,
  `DonGiaGioApDung` DECIMAL(10,0) NOT NULL,
  `TienNapTruoc` DECIMAL(12,0) NOT NULL,
  `TienGiuCho` DECIMAL(12,0) NOT NULL,
  `TongTienTamTinh` DECIMAL(12,0) NOT NULL,
  `TrangThaiPhien` VARCHAR(20) NOT NULL,
  `MaMay` VARCHAR(10) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `MaNguoiChoi` VARCHAR(15) CHARACTER SET ascii COLLATE ascii_bin NULL,
  `MaMayDangHoatDong` VARCHAR(10) CHARACTER SET ascii COLLATE ascii_bin GENERATED ALWAYS
  ↪  AS (CASE WHEN TrangThaiPhien='DangHoatDong' THEN MaMay ELSE NULL END) STORED,
  `MaNguoiChoiDangHoatDong` VARCHAR(15) CHARACTER SET ascii COLLATE ascii_bin GENERATED
  ↪  ALWAYS AS (CASE WHEN TrangThaiPhien='DangHoatDong' THEN MaNguoiChoi ELSE NULL END)
  ↪  STORED,
  PRIMARY KEY (`MaPhien`),
  CONSTRAINT `ck_phien_su_dung_0` CHECK (HinhThucDangNhap IN
  ↪  ('TraTruoc','TraSau','MienPhi')),
  CONSTRAINT `ck_phien_su_dung_1` CHECK (TrangThaiPhien IN
  ↪  ('DangHoatDong','DaKetThuc','DaHuy')),
  CONSTRAINT `ck_phien_su_dung_2` CHECK (DonGiaGioApDung>=0 AND TienNapTruoc>=0 AND
  ↪  TienGiuCho>=0 AND TongTienTamTinh>=0),
  CONSTRAINT `ck_phien_su_dung_3` CHECK (ThoiDiemKetThuc IS NULL OR
  ↪  ThoiDiemKetThuc>=ThoiDiemBatDau),
```

```
  CONSTRAINT `ck_phien_su_dung_4` CHECK ((TrangThaiPhien='DangHoatDong')=(ThoiDiemKetThuc IS NULL)),
  CONSTRAINT `ck_phien_su_dung_5` CHECK ((MaNguoiChoi IS NULL AND HinhThucDangNhap='TraTruoc') OR TienNapTruoc=0)
) ENGINE=InnoDB;

CREATE TABLE `BIEN_LAI_TIEN_MAY` (
  `MaBienLai` VARCHAR(20) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `MaPhien` VARCHAR(20) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `ThoiDiemLap` DATETIME NOT NULL,
  `ThoiDiemThanhToan` DATETIME NULL,
  `TongTienGio` DECIMAL(12,0) NOT NULL,
  `HinhThucThanhToan` VARCHAR(20) NOT NULL,
  `TienNhan` DECIMAL(12,0) NOT NULL,
  `TienHoan` DECIMAL(12,0) NOT NULL,
  `TrangThaiBienLai` VARCHAR(20) NOT NULL,
  `MaNhanVienThu` VARCHAR(15) CHARACTER SET ascii COLLATE ascii_bin NULL,
  PRIMARY KEY (`MaBienLai`),
  CONSTRAINT `uq_bien_lai_tien_may_0` UNIQUE (`MaPhien`),
  CONSTRAINT `ck_bien_lai_tien_may_0` CHECK (HinhThucThanhToan IN ('TienMat','ChuyenKhoan','TruVi','MienPhi')),
  CONSTRAINT `ck_bien_lai_tien_may_1` CHECK (TrangThaiBienLai IN ('ChuaThanhToan','DaThanhToan')),
  CONSTRAINT `ck_bien_lai_tien_may_2` CHECK (TongTienGio>=0 AND TienNhan>=0 AND TienHoan>=0)
) ENGINE=InnoDB;
```

DDL phân hệ dịch vụ

```
-- MySQL 8.4.13; baseline 23 tables. Added to unblock backend integration.

CREATE TABLE `NHOM_DICH_VU` (
  `MaNhomDV` VARCHAR(10) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `TenNhomDV` VARCHAR(50) NOT NULL,
  PRIMARY KEY (`MaNhomDV`),
  CONSTRAINT `uq_nhom_dich_vu_0` UNIQUE (`TenNhomDV`)
) ENGINE=InnoDB;

CREATE TABLE `DON_VI_TINH` (
  `MaDonVi` VARCHAR(10) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `TenDonVi` VARCHAR(30) NOT NULL,
  PRIMARY KEY (`MaDonVi`),
  CONSTRAINT `uq_don_vi_tinh_0` UNIQUE (`TenDonVi`)
) ENGINE=InnoDB;

CREATE TABLE `MAY_IN` (
  `MaMayIn` VARCHAR(10) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `TenMayIn` VARCHAR(50) NOT NULL,
  `ViTriDat` VARCHAR(100) NULL,
  PRIMARY KEY (`MaMayIn`),
  CONSTRAINT `uq_may_in_0` UNIQUE (`TenMayIn`)
) ENGINE=InnoDB;

CREATE TABLE `DICH_VU` (
  `MaDichVu` VARCHAR(10) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `TenDichVu` VARCHAR(100) NOT NULL,
  `GiaBanHienHanh` DECIMAL(10,0) NOT NULL,
  `SoLuongTon` INT NOT NULL,
  `TrangThaiKinhDoanh` VARCHAR(20) NOT NULL,
```

```
  `MaNhomDV` VARCHAR(10) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `MaDonVi` VARCHAR(10) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `MaMayIn` VARCHAR(10) CHARACTER SET ascii COLLATE ascii_bin NULL,
  PRIMARY KEY (`MaDichVu`),
  CONSTRAINT `ck_dich_vu_0` CHECK (GiaBanHienHanh>=0 AND SoLuongTon>=0),
  CONSTRAINT `ck_dich_vu_1` CHECK (TrangThaiKinhDoanh IN
  ↪ ('DangKinhDoanh','NgungKinhDoanh'))
) ENGINE=InnoDB;

CREATE TABLE `DON_GOI_DICH_VU` (
  `MaDonHang` VARCHAR(20) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `ThoiDiemDat` DATETIME NOT NULL,
  `KenhDat` VARCHAR(15) NOT NULL,
  `MaPhien` VARCHAR(20) CHARACTER SET ascii COLLATE ascii_bin NULL,
  `MaNhanVienXuLy` VARCHAR(15) CHARACTER SET ascii COLLATE ascii_bin NULL,
  `TongTien` DECIMAL(12,0) NOT NULL,
  `TrangThaiPhucVu` VARCHAR(20) NOT NULL,
  `TrangThaiTruocHuy` VARCHAR(20) NULL,
  `ThoiDiemHuy` DATETIME NULL,
  `TrangThaiThanhToan` VARCHAR(20) NOT NULL,
  `HinhThucThanhToan` VARCHAR(20) NULL,
  `ThoiDiemThanhToan` DATETIME NULL,
  `SoTienDaHoan` DECIMAL(12,0) NOT NULL DEFAULT 0,
  `ThoiDiemHoanTien` DATETIME NULL,
  PRIMARY KEY (`MaDonHang`),
  CONSTRAINT `ck_don_goi_dich_vu_0` CHECK (KenhDat IN ('TaiMay','TaiQuay')),
  CONSTRAINT `ck_don_goi_dich_vu_1` CHECK ((KenhDat='TaiMay' AND MaPhien IS NOT NULL) OR
  ↪ (KenhDat='TaiQuay' AND MaPhien IS NULL)),
  CONSTRAINT `ck_don_goi_dich_vu_2` CHECK (TrangThaiPhucVu IN
  ↪ ('MoiDat','DaTiepNhan','DangCheBien','DaGiao','DaHuy')),
  CONSTRAINT `ck_don_goi_dich_vu_3` CHECK (TrangThaiThanhToan IN
  ↪ ('ChuaThanhToan','DaThanhToan','DaHoanTien')),
  CONSTRAINT `ck_don_goi_dich_vu_4` CHECK (TongTien>=0 AND SoTienDaHoan>=0)
) ENGINE=InnoDB;

CREATE TABLE `CHI_TIET_GOI_DICH_VU` (
  `MaDonHang` VARCHAR(20) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `MaDichVu` VARCHAR(10) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `SoLuong` INT NOT NULL,
  `DonGiaBan` DECIMAL(10,0) NOT NULL,
  `ThanhTien` DECIMAL(12,0) GENERATED ALWAYS AS (SoLuong*DonGiaBan) STORED,
  PRIMARY KEY (`MaDonHang`,`MaDichVu`),
  CONSTRAINT `ck_chi_tiet_goi_dich_vu_0` CHECK (SoLuong>0 AND DonGiaBan>=0)
) ENGINE=InnoDB;
```

DDL phân hệ kho

```
-- MySQL 8.4.13; baseline 23 tables. Added to unblock backend integration.

CREATE TABLE `PHIEU_KHO` (
  `MaPhieuKho` VARCHAR(20) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `LoaiPhieu` VARCHAR(20) NOT NULL,
  `TrangThaiPhieu` VARCHAR(20) NOT NULL,
  `ThoiDiemLap` DATETIME NOT NULL,
  `ThoiDiemGhiSo` DATETIME NULL,
  `GhiChuLyDo` VARCHAR(255) NULL,
  `MaNhanVienLap` VARCHAR(15) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `MaDonHang` VARCHAR(20) CHARACTER SET ascii COLLATE ascii_bin NULL,
  PRIMARY KEY (`MaPhieuKho`),
```

```
  CONSTRAINT `uq_phieu_kho_0` UNIQUE (`MaDonHang`),
  CONSTRAINT `ck_phieu_kho_0` CHECK (LoaiPhieu IN
  ↪  ('NhapKho','XuatHuy','KiemKe','TaiPhanLoai')),
  CONSTRAINT `ck_phieu_kho_1` CHECK (TrangThaiPhieu IN ('Nhap','DaGhiSo','DaHuy')),
  CONSTRAINT `ck_phieu_kho_2` CHECK ((TrangThaiPhieu='DaGhiSo')=(ThoiDiemGhiSo IS NOT
  ↪  NULL))
) ENGINE=InnoDB;

CREATE TABLE `CHI_TIET_PHIEU_KHO` (
  `MaPhieuKho` VARCHAR(20) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `STTDong` INT NOT NULL,
  `MaDichVu` VARCHAR(10) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `SoLuong` INT NOT NULL,
  `SoLuongSoSach` INT NULL,
  `SoLuongThucTe` INT NULL,
  `SoLuongChenhLech` INT NOT NULL,
  `DonGiaVon` DECIMAL(10,0) NOT NULL,
  PRIMARY KEY (`MaPhieuKho`,`STTDong`),
  CONSTRAINT `uq_chi_tiet_phieu_kho_0` UNIQUE (`MaPhieuKho`,`MaDichVu`),
  CONSTRAINT `ck_chi_tiet_phieu_kho_0` CHECK (STTDong>0 AND SoLuong>=0 AND DonGiaVon>=0),
  CONSTRAINT `ck_chi_tiet_phieu_kho_1` CHECK (SoLuongSoSach IS NULL OR SoLuongSoSach>=0),
  CONSTRAINT `ck_chi_tiet_phieu_kho_2` CHECK (SoLuongThucTe IS NULL OR SoLuongThucTe>=0)
) ENGINE=InnoDB;
```

DDL phân hệ nhân sự và ví

```
-- MySQL 8.4.13; baseline 23 tables. Added to unblock backend integration.

CREATE TABLE `CA_TRUC` (
  `MaCa` VARCHAR(15) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `TenCa` VARCHAR(50) NOT NULL,
  `ThoiDiemBatDau` DATETIME NOT NULL,
  `ThoiDiemKetThuc` DATETIME NOT NULL,
  PRIMARY KEY (`MaCa`),
  CONSTRAINT `uq_ca_truc_0` UNIQUE (`TenCa`,`ThoiDiemBatDau`),
  CONSTRAINT `ck_ca_truc_0` CHECK (ThoiDiemKetThuc>ThoiDiemBatDau)
) ENGINE=InnoDB;

CREATE TABLE `PHAN_CONG_TRUC` (
  `MaNhanVien` VARCHAR(15) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `MaCa` VARCHAR(15) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `MaKhuVuc` VARCHAR(10) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `GhiChuNhiemVu` VARCHAR(255) NULL,
  PRIMARY KEY (`MaNhanVien`,`MaCa`,`MaKhuVuc`)
) ENGINE=InnoDB;

CREATE TABLE `GIAO_DICH_VI` (
  `MaGiaoDich` VARCHAR(20) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `MaYeuCau` VARCHAR(100) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `LoaiGiaoDich` VARCHAR(20) NOT NULL,
  `SoTien` DECIMAL(12,0) NOT NULL,
  `ThoiDiemGD` DATETIME NOT NULL,
  `TrangThaiGD` VARCHAR(20) NOT NULL,
  `PhuongThucGD` VARCHAR(20) NOT NULL,
  `MaNguoiChoi` VARCHAR(15) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  `MaNhanVienXuLy` VARCHAR(15) CHARACTER SET ascii COLLATE ascii_bin NULL,
  `MaDonHang` VARCHAR(20) CHARACTER SET ascii COLLATE ascii_bin NULL,
  `MaPhien` VARCHAR(20) CHARACTER SET ascii COLLATE ascii_bin NULL,
  `MaGDGoc` VARCHAR(20) CHARACTER SET ascii COLLATE ascii_bin NULL,
```

```
  `MaPhienThuThanhCong` VARCHAR(20) CHARACTER SET ascii COLLATE ascii_bin GENERATED ALWAYS AS (CASE WHEN LoaiGiaoDich='TruTienMay' AND TrangThaiGD='ThanhCong' THEN MaPhien ELSE NULL END) STORED,
  `MaDonHangThuThanhCong` VARCHAR(20) CHARACTER SET ascii COLLATE ascii_bin GENERATED ALWAYS AS (CASE WHEN LoaiGiaoDich='TruTienDichVu' AND TrangThaiGD='ThanhCong' THEN MaDonHang ELSE NULL END) STORED,
  `MaGDGocHoanThanhCong` VARCHAR(20) CHARACTER SET ascii COLLATE ascii_bin GENERATED ALWAYS AS (CASE WHEN LoaiGiaoDich='HoanTien' AND TrangThaiGD='ThanhCong' THEN MaGDGoc ELSE NULL END) STORED,
  PRIMARY KEY (`MaGiaoDich`),
  CONSTRAINT `uq_giao_dich_vi_0` UNIQUE (`MaYeuCau`),
  CONSTRAINT `ck_giao_dich_vi_0` CHECK (LoaiGiaoDich IN ('NapTien','TruTienMay','TruTienDichVu','HoanTien')),
  CONSTRAINT `ck_giao_dich_vi_1` CHECK (TrangThaiGD IN ('ChoXuLy','ThanhCong','ThatBai')),
  CONSTRAINT `ck_giao_dich_vi_2` CHECK (PhuongThucGD IN ('TienMat','ChuyenKhoan','NoiBo')),
  CONSTRAINT `ck_giao_dich_vi_3` CHECK (SoTien>0)
) ENGINE=InnoDB;
```

1.1.3 Khóa ngoại và chính sách toàn vẹn

Khóa ngoại bảo đảm các dòng chi tiết và chứng từ tham chiếu đối tượng tồn tại. Các tham chiếu lịch sử dùng RESTRICT khi việc xóa đối tượng cha có thể làm mất khả năng đối soát. CASCADE chỉ được sử dụng theo quan hệ đã định nghĩa trong DDL; không được hiểu là quyền tùy ý xóa chứng từ đã chốt.

Các khóa ngoại được áp dụng

```
-- Baseline foreign keys; install after all tables.
ALTER TABLE `TAI_KHOAN` ADD CONSTRAINT `fk_tai_khoan_0` FOREIGN KEY (`MaNhomND`) REFERENCES `NHOM_NGUOI_DUNG` (`MaNhomND`) ON DELETE RESTRICT;
ALTER TABLE `SO_DIEN_THOAI` ADD CONSTRAINT `fk_so_dien_thoai_0` FOREIGN KEY (`MaTaiKhoan`) REFERENCES `TAI_KHOAN` (`MaTaiKhoan`) ON DELETE CASCADE;
ALTER TABLE `NGUOI_CHOI` ADD CONSTRAINT `fk_nguoi_choi_0` FOREIGN KEY (`MaTaiKhoan`) REFERENCES `TAI_KHOAN` (`MaTaiKhoan`) ON DELETE CASCADE;
ALTER TABLE `NHAN_VIEN` ADD CONSTRAINT `fk_nhan_vien_0` FOREIGN KEY (`MaTaiKhoan`) REFERENCES `TAI_KHOAN` (`MaTaiKhoan`) ON DELETE CASCADE;
ALTER TABLE `NHAN_VIEN` ADD CONSTRAINT `fk_nhan_vien_1` FOREIGN KEY (`MaNguoiQuanLy`) REFERENCES `NHAN_VIEN` (`MaTaiKhoan`) ON DELETE SET NULL;
ALTER TABLE `DIA_CHI_NV` ADD CONSTRAINT `fk_dia_chi_nv_0` FOREIGN KEY (`MaTaiKhoan`) REFERENCES `NHAN_VIEN` (`MaTaiKhoan`) ON DELETE CASCADE;
ALTER TABLE `BANG_GIA` ADD CONSTRAINT `fk_bang_gia_0` FOREIGN KEY (`MaNhomND`) REFERENCES `NHOM_NGUOI_DUNG` (`MaNhomND`) ON DELETE CASCADE;
ALTER TABLE `BANG_GIA` ADD CONSTRAINT `fk_bang_gia_1` FOREIGN KEY (`MaNhomMay`) REFERENCES `NHOM_MAY` (`MaNhomMay`) ON DELETE CASCADE;
ALTER TABLE `MAY_TRAM` ADD CONSTRAINT `fk_may_tram_0` FOREIGN KEY (`MaNhomMay`) REFERENCES `NHOM_MAY` (`MaNhomMay`) ON DELETE RESTRICT;
ALTER TABLE `MAY_TRAM` ADD CONSTRAINT `fk_may_tram_1` FOREIGN KEY (`MaKhuVuc`) REFERENCES `KHU_VUC` (`MaKhuVuc`) ON DELETE RESTRICT;
ALTER TABLE `PHIEN_SU_DUNG` ADD CONSTRAINT `fk_phien_su_dung_0` FOREIGN KEY (`MaMay`) REFERENCES `MAY_TRAM` (`MaMay`) ON DELETE RESTRICT;
ALTER TABLE `PHIEN_SU_DUNG` ADD CONSTRAINT `fk_phien_su_dung_1` FOREIGN KEY (`MaNguoiChoi`) REFERENCES `NGUOI_CHOI` (`MaTaiKhoan`) ON DELETE RESTRICT;
ALTER TABLE `BIEN_LAI_TIEN_MAY` ADD CONSTRAINT `fk_bien_lai_tien_may_0` FOREIGN KEY (`MaPhien`) REFERENCES `PHIEN_SU_DUNG` (`MaPhien`) ON DELETE RESTRICT;
ALTER TABLE `BIEN_LAI_TIEN_MAY` ADD CONSTRAINT `fk_bien_lai_tien_may_1` FOREIGN KEY (`MaNhanVienThu`) REFERENCES `NHAN_VIEN` (`MaTaiKhoan`) ON DELETE RESTRICT;
```

```
16 ALTER TABLE `PHAN_CONG_TRUC` ADD CONSTRAINT `fk_phan_cong_truc_0` FOREIGN KEY
   ↪  (`MaNhanVien`) REFERENCES `NHAN_VIEN` (`MaTaiKhoan`) ON DELETE RESTRICT;
17 ALTER TABLE `PHAN_CONG_TRUC` ADD CONSTRAINT `fk_phan_cong_truc_1` FOREIGN KEY (`MaCa`)
   ↪  REFERENCES `CA_TRUC` (`MaCa`) ON DELETE RESTRICT;
18 ALTER TABLE `PHAN_CONG_TRUC` ADD CONSTRAINT `fk_phan_cong_truc_2` FOREIGN KEY
   ↪  (`MaKhuVuc`) REFERENCES `KHU_VUC` (`MaKhuVuc`) ON DELETE RESTRICT;
19 ALTER TABLE `DICH_VU` ADD CONSTRAINT `fk_dich_vu_0` FOREIGN KEY (`MaNhomDV`) REFERENCES
   ↪  `NHOM_DICH_VU` (`MaNhomDV`) ON DELETE RESTRICT;
20 ALTER TABLE `DICH_VU` ADD CONSTRAINT `fk_dich_vu_1` FOREIGN KEY (`MaDonVi`) REFERENCES
   ↪  `DON_VI_TINH` (`MaDonVi`) ON DELETE RESTRICT;
21 ALTER TABLE `DICH_VU` ADD CONSTRAINT `fk_dich_vu_2` FOREIGN KEY (`MaMayIn`) REFERENCES
   ↪  `MAY_IN` (`MaMayIn`) ON DELETE SET NULL;
22 ALTER TABLE `DON_GOI_DICH_VU` ADD CONSTRAINT `fk_don_goi_dich_vu_0` FOREIGN KEY
   ↪  (`MaPhien`) REFERENCES `PHIEN_SU_DUNG` (`MaPhien`) ON DELETE RESTRICT;
23 ALTER TABLE `DON_GOI_DICH_VU` ADD CONSTRAINT `fk_don_goi_dich_vu_1` FOREIGN KEY
   ↪  (`MaNhanVienXuLy`) REFERENCES `NHAN_VIEN` (`MaTaiKhoan`) ON DELETE RESTRICT;
24 ALTER TABLE `CHI_TIET_GOI_DICH_VU` ADD CONSTRAINT `fk_chi_tiet_goi_dich_vu_0` FOREIGN KEY
   ↪  (`MaDonHang`) REFERENCES `DON_GOI_DICH_VU` (`MaDonHang`) ON DELETE CASCADE;
25 ALTER TABLE `CHI_TIET_GOI_DICH_VU` ADD CONSTRAINT `fk_chi_tiet_goi_dich_vu_1` FOREIGN KEY
   ↪  (`MaDichVu`) REFERENCES `DICH_VU` (`MaDichVu`) ON DELETE RESTRICT;
26 ALTER TABLE `PHIEU_KHO` ADD CONSTRAINT `fk_phieu_kho_0` FOREIGN KEY (`MaNhanVienLap`)
   ↪  REFERENCES `NHAN_VIEN` (`MaTaiKhoan`) ON DELETE RESTRICT;
27 ALTER TABLE `PHIEU_KHO` ADD CONSTRAINT `fk_phieu_kho_1` FOREIGN KEY (`MaDonHang`)
   ↪  REFERENCES `DON_GOI_DICH_VU` (`MaDonHang`) ON DELETE RESTRICT;
28 ALTER TABLE `CHI_TIET_PHIEU_KHO` ADD CONSTRAINT `fk_chi_tiet_phieu_kho_0` FOREIGN KEY
   ↪  (`MaPhieuKho`) REFERENCES `PHIEU_KHO` (`MaPhieuKho`) ON DELETE CASCADE;
29 ALTER TABLE `CHI_TIET_PHIEU_KHO` ADD CONSTRAINT `fk_chi_tiet_phieu_kho_1` FOREIGN KEY
   ↪  (`MaDichVu`) REFERENCES `DICH_VU` (`MaDichVu`) ON DELETE RESTRICT;
30 ALTER TABLE `GIAO_DICH_VI` ADD CONSTRAINT `fk_giao_dich_vi_0` FOREIGN KEY (`MaNguoiChoi`)
   ↪  REFERENCES `NGUOI_CHOI` (`MaTaiKhoan`) ON DELETE RESTRICT;
31 ALTER TABLE `GIAO_DICH_VI` ADD CONSTRAINT `fk_giao_dich_vi_1` FOREIGN KEY
   ↪  (`MaNhanVienXuLy`) REFERENCES `NHAN_VIEN` (`MaTaiKhoan`) ON DELETE RESTRICT;
32 ALTER TABLE `GIAO_DICH_VI` ADD CONSTRAINT `fk_giao_dich_vi_2` FOREIGN KEY (`MaDonHang`)
   ↪  REFERENCES `DON_GOI_DICH_VU` (`MaDonHang`) ON DELETE RESTRICT;
33 ALTER TABLE `GIAO_DICH_VI` ADD CONSTRAINT `fk_giao_dich_vi_3` FOREIGN KEY (`MaPhien`)
   ↪  REFERENCES `PHIEN_SU_DUNG` (`MaPhien`) ON DELETE RESTRICT;
34 ALTER TABLE `GIAO_DICH_VI` ADD CONSTRAINT `fk_giao_dich_vi_4` FOREIGN KEY (`MaGDGoc`)
   ↪  REFERENCES `GIAO_DICH_VI` (`MaGiaoDich`) ON DELETE RESTRICT;
```

MySQL SQL Client

Máy chủ MySQL 8.4.13 · Kết nối local · Phiên kiểm chứng BTL2 L09 Nhóm 10

img_01_dbms_tables

```
SHOW TABLES;
SELECT VERSION() AS MySQLVersion, COUNT(*) AS SoBang FROM information_schema.TABLES WHERE TABLE_SCHEMA=DATABASE();
```

Thực thi SQL btl2_evidence_a62ff5712250 · Hoàn tất

23 dòng · 16.34 ms

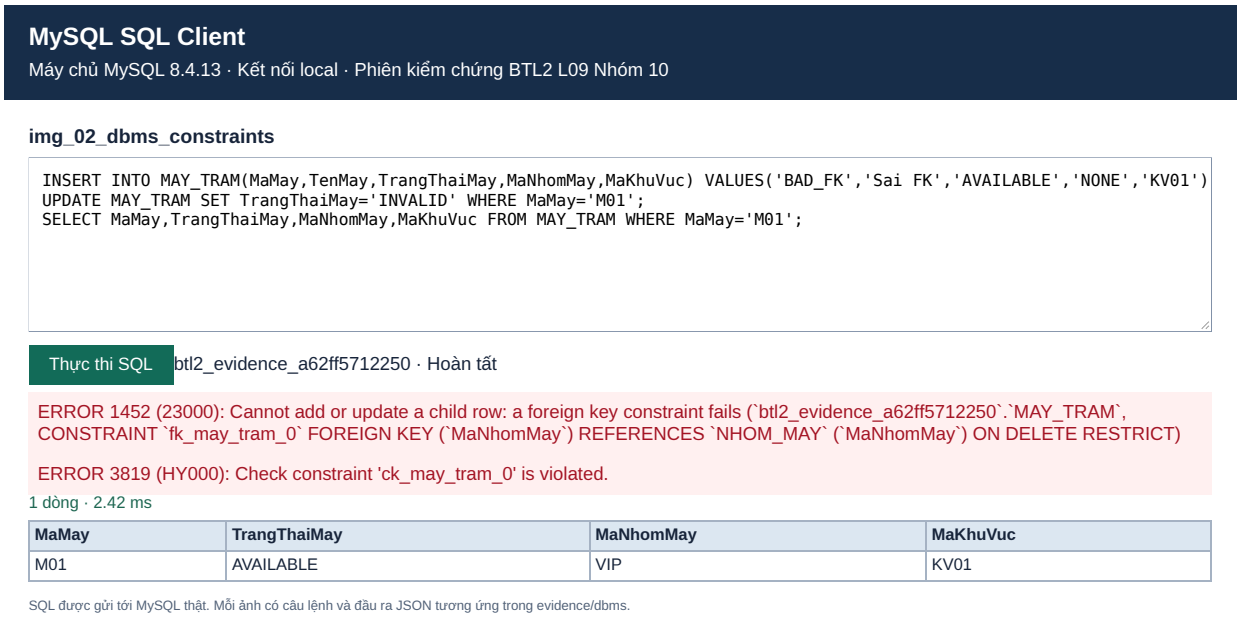
Tables_in_btl2_evidence_a62ff5712250
BANG_GIA
BIEN_LAI_TIEN_MAY
CA_TRUC
CHI_TIET_GOI_DICH_VU
CHI_TIET_PHIEU_KHO
DIA_CHI_NV
DICH_VU
DON_GOI_DICH_VU
DON_VI_TINH
GIAO_DICH_VI
KHU_VUC
MAY_IN
MAY_TRAM
NGUOI_CHOI
NHAN_VIEN
NHOM_DICH_VU
NHOM_MAY
NHOM_NGUOI_DUNG
PHAN_CONG_TRUC
PHIEN_SU_DUNG
PHIEU_KHO
SO_DIEN_THOAI
TAI_KHOAN

1 dòng · 5.07 ms

MySQLVersion	SoBang
8.4.13	23

SQL được gửi tới MySQL thật. Mỗi ảnh có câu lệnh và đầu ra JSON tương ứng trong evidence/dbms.

Hình 1.1. Kết quả DBMS minh họa câu lệnh tạo bảng thực thi thành công (23 bảng)



Hình 1.2. Kết quả DBMS minh họa các ràng buộc khóa ngoại và CHECK được áp dụng

1.2 Câu lệnh tạo chỉ mục

1.2.1 Mô tả lý do cần đánh chỉ mục

Ở màn hình gọi món tại máy hoặc bán tại quầy, người dùng lọc dịch vụ đang kinh doanh theo nhóm và khoảng giá, sau đó xem giá từ cao xuống thấp. Yêu cầu này xuất phát từ chức năng xem thực đơn phục vụ khách và thu ngân. Truy vấn Q-P12-02 vì vậy là cơ sở lựa chọn chỉ mục nhiều cột, không đánh chỉ mục tách rời khỏi nhu cầu sử dụng thực tế.

Câu lệnh tạo chỉ mục: ix_dich_vu_filter

```
CREATE UNIQUE INDEX uq_phien_may_active ON PHIEN_SU_DUNG(MaMayDangHoatDong);
CREATE UNIQUE INDEX uq_phien_nguoi_active ON PHIEN_SU_DUNG(MaNguoiChoiDangHoatDong);
CREATE UNIQUE INDEX uq_vi_thu_phien ON GIAO_DICH_VI(MaPhienThuThanhCong);
CREATE UNIQUE INDEX uq_vi_thu_don ON GIAO_DICH_VI(MaDonHangThuThanhCong);
CREATE UNIQUE INDEX uq_vi_hoan_goc ON GIAO_DICH_VI(MaGDGocHoanThanhCong);
CREATE INDEX ix_don_thoi_diem ON DON_GOI_DICH_VU(ThoiDiemDat);
CREATE INDEX ix_bien_lai_thanh_toan ON BIEN_LAI_TIEN_MAY(ThoiDiemThanhToan);
CREATE INDEX ix_vi_nguoi_thoi_diem ON GIAO_DICH_VI(MaNguoiChoi,ThoiDiemGD);
CREATE INDEX ix_ca_khoang ON CA_TRUC(ThoiDiemBatDau,ThoiDiemKetThuc);
CREATE INDEX ix_dich_vu_filter ON DICH_VU(MaNhomDV, TrangThaiKinhDoanh, GiaBanHienHanh,
↪  MaDichVu);
```

1.2.2 Minh họa hiệu quả truy vấn trước và sau khi đánh chỉ mục

Phép đo dùng database riêng với 20.000 dòng dịch vụ, 10 nhóm và hai đơn vị tính. Cùng một câu SELECT được chạy 20 lượt trước và sau khi thêm `ix_dich_vu_filter`. Cả hai giai đoạn đều lấy hết tập kết quả. Số đo phụ thuộc máy chạy, cache và tải tại thời điểm đo; đây là bằng chứng cho tập dữ liệu cụ thể, không phải cam kết hiệu năng trong mọi môi trường.

Bảng 1.2. So sánh hiệu năng truy vấn trước và sau khi đánh chỉ mục

Chỉ tiêu	Trước	Sau
Thời gian trung bình 20 lượt (ms)	267.313	68.820
Số dòng kết quả	949	949
Có bước Sort	Có	Có

Hiệu quả tăng tốc đạt **3.88 lần** (từ 267.3ms xuống 68.8ms). Kế hoạch thực thi dưới đây minh họa chi tiết đường dẫn truy xuất dữ liệu từ Table Scan sang Index Range Scan:

Kế hoạch thực thi EXPLAIN ANALYZE trước khi tạo chỉ mục

```
-> Sort: d.GiaBanHienHanh DESC, d.MaDichVu  (actual time=249..250 rows=949 loops=1)
    -> Stream results  (cost=1.15 rows=0.1) (actual time=1.97..247 rows=949 loops=1)
        -> Nested loop inner join  (cost=1.15 rows=0.1) (actual time=1.96..246 rows=949
        ↪  loops=1)
            -> Table scan on dvt  (cost=0.45 rows=2) (actual time=0.538..0.549 rows=2
            ↪  loops=1)
            -> Filter: ((d.MaNhomDV = 'NDV01') and (d.TrangThaiKinhDoanh =
            ↪  'DangKinhDoanh') and (d.GiaBanHienHanh between 10000 and 50000))
            ↪  (cost=0.253 rows=0.05) (actual time=64.9..122 rows=474 loops=2)
                -> Index lookup on d using MaDonVi (MaDonVi=dvt.MaDonVi)  (cost=0.253
                ↪  rows=1) (actual time=0.141..115 rows=10000 loops=2)
```

Kế hoạch thực thi EXPLAIN ANALYZE sau khi tạo chỉ mục

```
-> Sort: d.GiaBanHienHanh DESC, d.MaDichVu  (actual time=19.7..19.9 rows=949 loops=1)
    -> Stream results  (cost=618 rows=949) (actual time=17.2..17.9 rows=949 loops=1)
        -> Inner hash join (dvt.MaDonVi = d.MaDonVi)  (cost=618 rows=949) (actual
        ↪  time=17.1..17.4 rows=949 loops=1)
            -> Table scan on dvt  (cost=956e-6 rows=2) (actual time=0.016..0.0339 rows=2
            ↪  loops=1)
            -> Hash
                -> Index range scan on d using ix_dich_vu_filter over (MaNhomDV = 'NDV01'
                ↪  AND TrangThaiKinhDoanh = 'DangKinhDoanh' AND 10000 <= GiaBanHienHanh
                ↪  <= 50000), with index condition: ((d.MaNhomDV = 'NDV01') and
                ↪  (d.TrangThaiKinhDoanh = 'DangKinhDoanh') and (d.GiaBanHienHanh
                ↪  between 10000 and 50000))  (cost=427 rows=949) (actual
                ↪  time=2.18..16.3 rows=949 loops=1)
```

MySQL SQL Client

Máy chủ MySQL 8.4.13 · Kết nối local · Phiên kiểm chứng BTL2 L09 Nhóm 10

index_before

```
EXPLAIN ANALYZE SELECT
    d.`MaDichVu`,
    d.`TenDichVu`,
    ndv.`TenNhomDV`,
    dvt.`TenDonVi`,
    d.`GiaBanHienHanh`,
    d.`SoLuongTon`
FROM `DICH_VU` d
```

Thực thi SQL btl2_benchmark_95332505a18a · Hoàn tất

1 dòng · 255.54 ms

EXPLAIN
-> Sort: d.GiaBanHienHanh DESC, d.MaDichVu (actual time=248..248 rows=949 loops=1) -> Stream results (cost=1.15 rows=0.1) (actual time=0.174..245 rows=949 loops=1) -> Nested loop inner join (cost=1.15 rows=0.1) (actual time=0.166..244 rows=949 loops=1) -> Table scan on dvt (cost=0.45 rows=2) (actual time=0.016..0.028 rows=2 loops=1) -> Filter: ((d.MaNhomDV = 'NDV01') and (d.TrangThaiKinhDoanh = 'DangKinhDoanh') and (d.GiaBanHienHanh between 10000 and 50000)) (cost=0.253 rows=0.05) (actual time=51.9..122 rows=474 loops=2) -> Index lookup on d using MaDonVi (MaDonVi=dvt.MaDonVi) (cost=0.253 rows=1) (actual time=0.123..114 rows=10000 loops=2)

SQL được gửi tới MySQL thật. Mỗi ảnh có câu lệnh và đầu ra JSON tương ứng trong evidence/dbms.

Hình 1.3. Kết quả DBMS hiển thị EXPLAIN ANALYZE trước khi tạo chỉ mục

MySQL SQL Client

Máy chủ MySQL 8.4.13 · Kết nối local · Phiên kiểm chứng BTL2 L09 Nhóm 10

index_after

```
EXPLAIN ANALYZE SELECT
    d.`MaDichVu`,
    d.`TenDichVu`,
    ndv.`TenNhomDV`,
    dvt.`TenDonVi`,
    d.`GiaBanHienHanh`,
    d.`SoLuongTon`
FROM `DICH_VU` d
```

Thực thi SQL btl2_benchmark_95332505a18a · Hoàn tất

1 dòng · 35.15 ms

EXPLAIN
-> Sort: d.GiaBanHienHanh DESC, d.MaDichVu (actual time=27.9..28.1 rows=949 loops=1) -> Stream results (cost=618 rows=949) (actual time=24.1..25.6 rows=949 loops=1) -> Inner hash join (dvt.MaDonVi = d.MaDonVi) (cost=618 rows=949) (actual time=24.1..24.6 rows=949 loops=1) -> Table scan on dvt (cost=956e-6 rows=2) (actual time=0.0282..0.0524 rows=2 loops=1) -> Hash -> Index range scan on d using ix_dich_vu_filter over (MaNhomDV = 'NDV01' AND TrangThaiKinhDoanh = 'DangKinhDoanh' AND 10000 <= GiaBanHienHanh <= 50000), with index condition: ((d.MaNhomDV = 'NDV01') and (d.TrangThaiKinhDoanh = 'DangKinhDoanh') and (d.GiaBanHienHanh between 10000 and 50000)) (cost=427 rows=949) (actual time=2.37..22.9 rows=949 loops=1)

SQL được gửi tới MySQL thật. Mỗi ảnh có câu lệnh và đầu ra JSON tương ứng trong evidence/dbms.

Hình 1.4. Kết quả DBMS hiển thị EXPLAIN ANALYZE sau khi tạo chỉ mục

1.3 Truy vấn dữ liệu

Yêu cầu: 5 câu truy vấn độc lập trên các bảng dữ liệu khác nhau, kèm dữ liệu mẫu và kết quả thực thi trên DBMS. Hai câu đầu có SELECT, FROM, WHERE, ORDER BY; hai câu tiếp theo có GROUP BY và AGGREGATE; câu cuối cùng lồng 2 cấp và AGGREGATE. Các bảng dữ liệu mẫu chi tiết được đối chiếu thêm tại Phụ lục C.

1.3.1 Câu truy vấn 1: Danh sách máy theo khu vực (Q-P12-01)

Mô tả: Dùng cho màn hình sơ đồ máy của thu ngân để kiểm tra các máy có thể sử dụng tại KV01, hiển thị tên máy, nhóm máy và đơn giá giờ; sắp xếp theo vị trí hàng, cột.

Câu lệnh SELECT - Truy vấn 1 (Q-P12-01)

```
SELECT
    m.`MaMay`,
    m.`TenMay`,
    m.`TrangThaiMay`,
    k.`TenKhuVuc`,
    nm.`TenNhomMay`,
    nm.`DonGiaGioMacDinh`,
    m.`ViTriHang`,
    m.`ViTriCot`
FROM `MAY_TRAM` m
JOIN `KHU_VUC` k ON m.`MaKhuVuc` = k.`MaKhuVuc`
JOIN `NHOM_MAY` nm ON m.`MaNhomMay` = nm.`MaNhomMay`
WHERE m.`TrangThaiMay` IN ('AVAILABLE', 'ONLINE')
  AND k.`MaKhuVuc` = 'KV01'
ORDER BY m.`ViTriHang` ASC, m.`ViTriCot` ASC;
```

Bảng 1.3. Bảng dữ liệu mẫu và kết quả thực thi Truy vấn 1

Mã máy	Tên máy	Trạng thái	Nhóm máy	Giá giờ
M01	Máy 01	AVAILABLE	Máy VIP	15000
M02	Máy 02	AVAILABLE	Máy VIP	15000
M04	Máy 04	AVAILABLE	Máy VIP	15000



Hình 1.5. Kết quả màn hình hiển thị từ DBMS minh họa thực thi Truy vấn 1

1.3.2 Câu truy vấn 2: Lọc thực đơn theo nhóm và giá (Q-P12-02)

Mô tả: Dùng cho màn hình gọi món của khách hoặc bán tại quầy của thu ngân để tìm các món thuộc nhóm NDV01 có giá từ 10.000 đến 50.000đ; sắp theo giá giảm dần rồi theo mã.

Câu lệnh SELECT - Truy vấn 2 (Q-P12-02)

```
SELECT
    d.`MaDichVu`,
    d.`TenDichVu`,
    ndv.`TenNhomDV`,
    dvt.`TenDonVi`,
    d.`GiaBanHienHanh`,
    d.`SoLuongTon`
FROM `DICH_VU` d
JOIN `NHOM_DICH_VU` ndv ON d.`MaNhomDV` = ndv.`MaNhomDV`
JOIN `DON_VI_TINH` dvt ON d.`MaDonVi` = dvt.`MaDonVi`
WHERE d.`TrangThaiKinhDoanh` = 'DangKinhDoanh'
  AND d.`MaNhomDV` = 'NDV01'
  AND d.`GiaBanHienHanh` BETWEEN 10000 AND 50000
ORDER BY d.`GiaBanHienHanh` DESC, d.`MaDichVu` ASC;
```

Bảng 1.4. Bảng dữ liệu mẫu và kết quả thực thi Truy vấn 2

Mã DV	Tên dịch vụ	Đơn vị	Giá bán	Tồn
DV03	Cơm chiên trứng	Phần	25000	30
DV01	Mì trứng	Phần	15000	50



Hình 1.6. Kết quả màn hình hiển thị từ DBMS minh họa thực thi Truy vấn 2

1.3.3 Câu truy vấn 3: Tổng hợp đơn đã thanh toán (Q-P12-03)

Mô tả: Dùng cho màn hình báo cáo ca của quản lý để thống kê số đơn và doanh số theo ngày đặt và kênh đặt; có nhóm theo ngày, kênh và tính tổng tiền, đếm số đơn.

Câu lệnh SELECT - Truy vấn 3 (Q-P12-03)

```
SELECT
    DATE(d.`ThoiDiemDat`) AS `NgayDat`,
    d.`KenhDat`,
    COUNT(d.`MaDonHang`) AS `SoLuongDon`,
    SUM(d.`TongTien`) AS `TongTienDonHang`
FROM `DON_GOI_DICH_VU` d
WHERE d.`TrangThaiThanhToan` = 'DaThanhToan'
  AND d.`ThoiDiemDat` >= '2026-10-01 00:00:00'
  AND d.`ThoiDiemDat` <= '2026-11-30 23:59:59'
GROUP BY DATE(d.`ThoiDiemDat`), d.`KenhDat`
ORDER BY `NgayDat` DESC, `TongTienDonHang` DESC;
```

Bảng 1.5. Bảng dữ liệu mẫu và kết quả thực thi Truy vấn 3

Ngày đặt	Kênh đặt	Số đơn	Tổng tiền
2026-10-21	TaiQuay	1	50000
2026-10-20	TaiQuay	1	52000



Hình 1.7. Kết quả màn hình hiển thị từ DBMS minh họa thực thi Truy vấn 3

1.3.4 Câu truy vấn 4: Số ca và thời lượng trực (Q-P12-04)

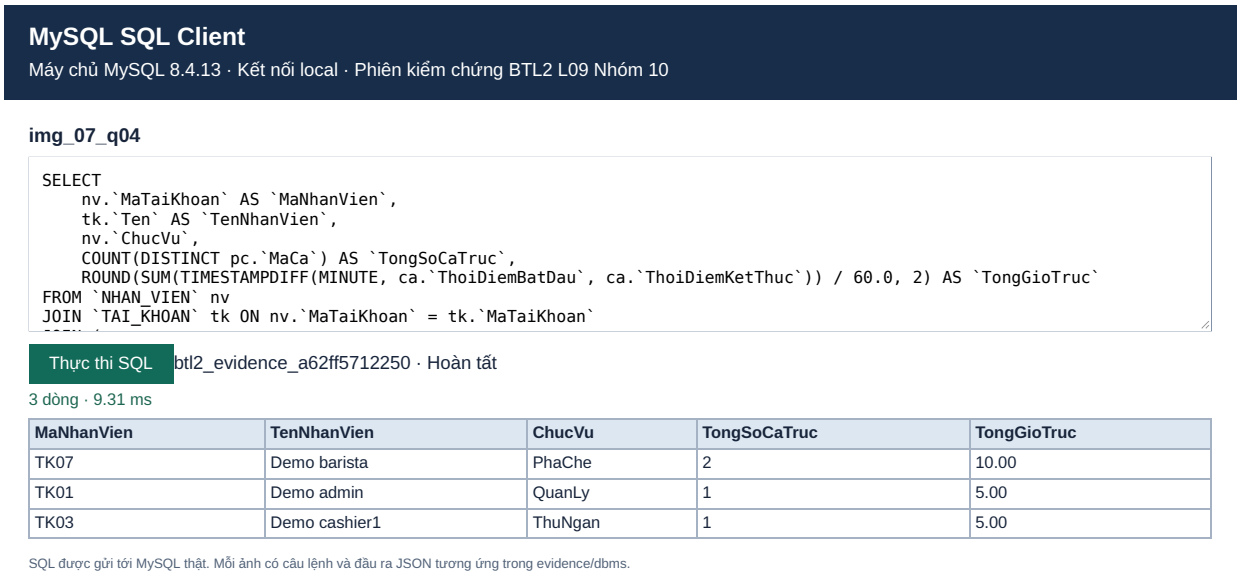
Mô tả: Dùng cho màn hình chấm công và phân ca để thống kê tổng số ca và tổng thời lượng trực của nhân viên; loại bỏ trùng lặp mã ca trước khi cộng tổng.

Câu lệnh SELECT - Truy vấn 4 (Q-P12-04)

```sql
SELECT
    nv.`MaTaiKhoan` AS `MaNhanVien`,
    tk.`Ten` AS `TenNhanVien`,
    nv.`ChucVu`,
    COUNT(DISTINCT pc.`MaCa`) AS `TongSoCaTruc`,
    ROUND(SUM(TIMESTAMPDIFF(MINUTE, ca.`ThoiDiemBatDau`, ca.`ThoiDiemKetThuc`)) / 60.0,
    ↪  2) AS `TongGioTruc`
FROM `NHAN_VIEN` nv
JOIN `TAI_KHOAN` tk ON nv.`MaTaiKhoan` = tk.`MaTaiKhoan`
JOIN (
    SELECT DISTINCT `MaNhanVien`, `MaCa`
    FROM `PHAN_CONG_TRUC`
) pc ON nv.`MaTaiKhoan` = pc.`MaNhanVien`
JOIN `CA_TRUC` ca ON pc.`MaCa` = ca.`MaCa`
WHERE ca.`ThoiDiemBatDau` >= '2026-10-01 00:00:00'
  AND ca.`ThoiDiemKetThuc` <= '2026-11-30 23:59:59'
GROUP BY nv.`MaTaiKhoan`, tk.`Ten`, nv.`ChucVu`
ORDER BY `TongGioTruc` DESC, `TongSoCaTruc` DESC;
```

Bảng 1.6. Bảng dữ liệu mẫu và kết quả thực thi Truy vấn 4

Mã NV	Tên nhân viên	Số ca	Số giờ
TK07	Demo barista	2	10.00
TK01	Demo admin	1	5.00
TK03	Demo cashier1	1	5.00



Hình 1.8. Kết quả màn hình hiển thị từ DBMS minh họa thực thi Truy vấn 4

1.3.5 Câu truy vấn 5: Hội viên có số dư vượt mức trung bình (Q-P12-05)

Mô tả: Dùng cho màn hình chăm sóc khách hàng VIP để tìm hội viên có số dư ví cao hơn mức số dư trung bình của nhóm hội viên từng phát sinh giao dịch ví thành công.

Câu lệnh SELECT - Truy vấn 5 (Q-P12-05)

```sql
SELECT
    nc.`MaTaiKhoan` AS `MaHoiVien`,
    tk.`Ten` AS `TenHoiVien`,
    nc.`SoCCCD`,
    nc.`SoDuVi`,
    nc.`NgayDangKy`
FROM `NGUOI_CHOI` nc
JOIN `TAI_KHOAN` tk ON nc.`MaTaiKhoan` = tk.`MaTaiKhoan`
WHERE nc.`SoDuVi` > (
    SELECT AVG(sub_nc.`SoDuVi`)
    FROM `NGUOI_CHOI` sub_nc
    WHERE sub_nc.`MaTaiKhoan` IN (
        SELECT DISTINCT gd.`MaNguoiChoi`
        FROM `GIAO_DICH_VI` gd
        WHERE gd.`TrangThaiGD` = 'ThanhCong'
          AND gd.`ThoiDiemGD` >= '2026-10-01 00:00:00'
          AND gd.`ThoiDiemGD` <= '2026-11-30 23:59:59'
    )
)
ORDER BY nc.`SoDuVi` DESC, nc.`MaTaiKhoan` ASC;
```

Bảng 1.7. Bảng dữ liệu mẫu và kết quả thực thi Truy vấn 5

Mã hội viên	Tên hội viên	Số dư ví	Ngày đăng ký
TK09	Demo player3	250000	2026-10-01



Hình 1.9. Kết quả màn hình hiển thị từ DBMS minh họa thực thi Truy vấn 5

PHẦN 2

THỦ TỤC – HÀM - TRIGGER

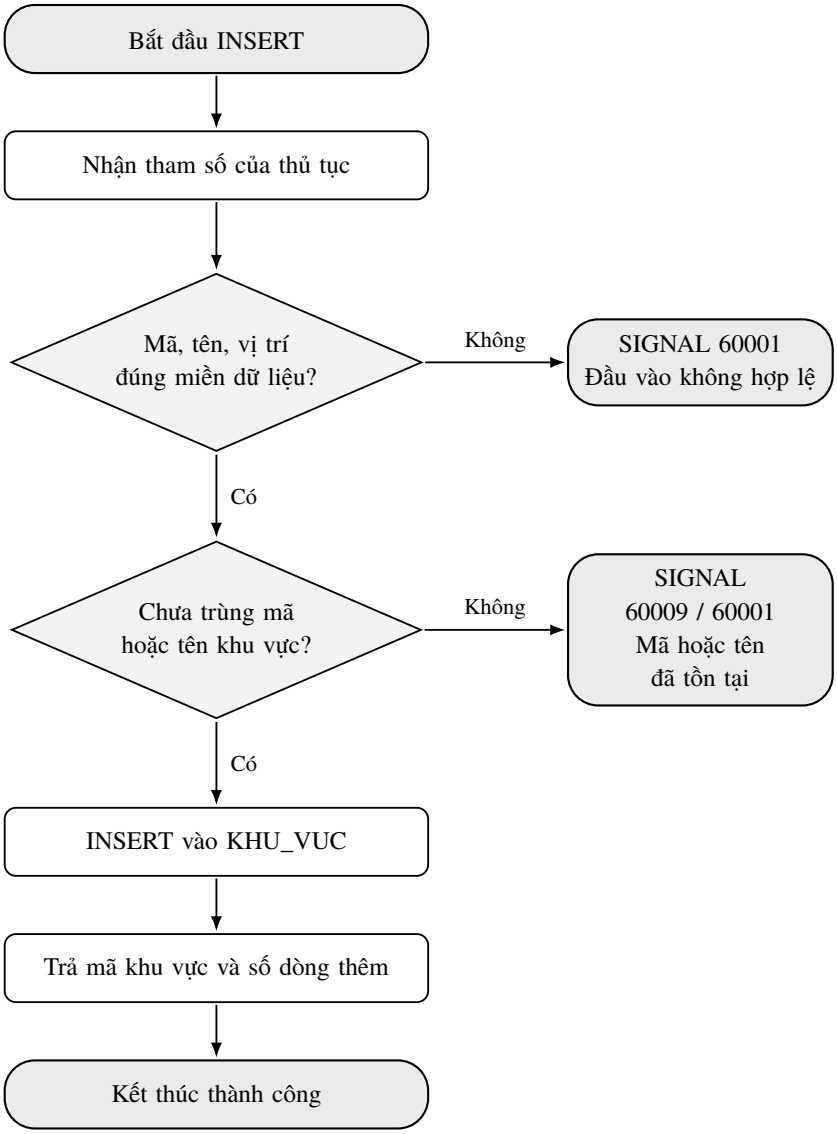
2.1 Thủ tục INSERT / UPDATE / DELETE dữ liệu vào 1 bảng dữ liệu

2.1.1 Nguyên tắc xử lý và bảng dữ liệu chính

Bảng dữ liệu chính liên quan: `KHU_VUC`. Bảng dữ liệu phụ liên quan: `MAY_TRAM`, `PHAN_CONG_TRUC`. Thủ tục kiểm tra đầy đủ điều kiện ràng buộc nghiệp vụ và xử lý ngoại lệ có mã lỗi rõ ràng. Mỗi thủ tục trả về mã khu vực, tên thao tác và số dòng bị tác động.

2.1.2 Thủ tục INSERT: Thêm khu vực (sp_khu_vuc_insert)

Mô tả: Thêm khu vực mới trong màn hình quản trị mặt bằng quán. Kiểm tra mã không rỗng, tên không trùng. Input: `p_ma_khu_vuc`, `p_ten_khu_vuc`, `p_mo_ta_vi_tri`. Output: `ma_khu_vuc`, `thao_tac`, `so_dong_tac_dong`.



Hình 2.1. Lưu đồ minh họa thao tác xử lý thủ tục INSERT

Câu lệnh tạo thủ tục INSERT

```
DELIMITER $$

CREATE PROCEDURE `sp_khu_vuc_insert`(
    IN pMaKhuVuc VARCHAR(50),
    IN pTenKhuVuc VARCHAR(100),
    IN pMoTaViTri VARCHAR(255)
)
proc_label: BEGIN
    DECLARE vMa VARCHAR(10);
    DECLARE vTen VARCHAR(60);

    -- 1. Kiểm tra tham số đầu vào
    IF pMaKhuVuc IS NULL OR TRIM(pMaKhuVuc) = '' OR CHAR_LENGTH(TRIM(pMaKhuVuc)) > 10
    ↪  THEN
        SIGNAL SQLSTATE '45000' SET MYSQL_ERRNO = 60001, MESSAGE_TEXT = 'INVALID_INPUT:
        ↪  MaKhuVuc khong hop le';
    END IF;
```

```
    IF pTenKhuVuc IS NULL OR TRIM(pTenKhuVuc) = '' OR CHAR_LENGTH(TRIM(pTenKhuVuc)) > 60 THEN
        SIGNAL SQLSTATE '45000' SET MYSQL_ERRNO = 60001, MESSAGE_TEXT = 'INVALID_INPUT: TenKhuVuc khong hop le';
    END IF;

    SET vMa = TRIM(pMaKhuVuc);
    SET vTen = TRIM(pTenKhuVuc);

    -- 2. Kiểm tra trùng lặp khóa chính
    IF EXISTS (SELECT 1 FROM `KHU_VUC` WHERE `MaKhuVuc` = vMa) THEN
        SIGNAL SQLSTATE '45000' SET MYSQL_ERRNO = 60009, MESSAGE_TEXT = 'IDEMPOTENCY_CONFLICT: MaKhuVuc da ton tai';
    END IF;

    -- 3. Kiểm tra trùng lặp tên khu vực (UNIQUE)
    IF EXISTS (SELECT 1 FROM `KHU_VUC` WHERE `TenKhuVuc` = vTen) THEN
        SIGNAL SQLSTATE '45000' SET MYSQL_ERRNO = 60001, MESSAGE_TEXT = 'INVALID_INPUT: TenKhuVuc da ton tai';
    END IF;

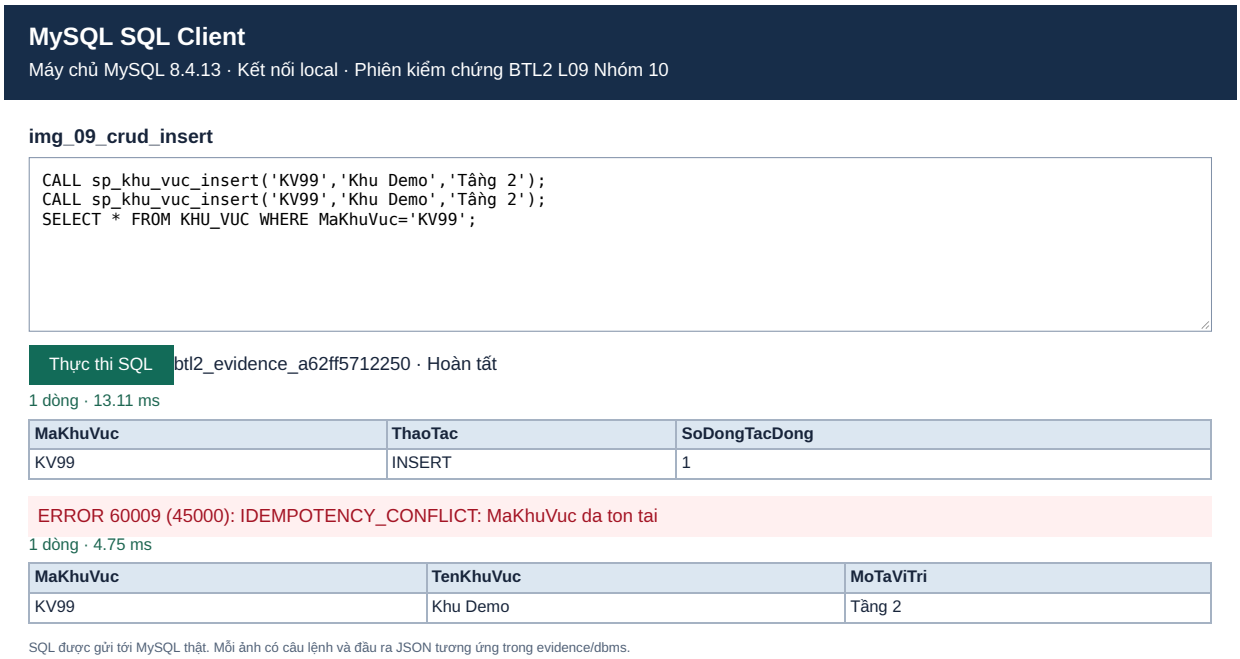
    -- 4. Thực hiện thêm mới
    INSERT INTO `KHU_VUC` (`MaKhuVuc`, `TenKhuVuc`, `MoTaViTri`)
    VALUES (vMa, vTen, pMoTaViTri);

    -- 5. Trả kết quả thực thi
    SELECT vMa AS `MaKhuVuc`, 'INSERT' AS `ThaoTac`, ROW_COUNT() AS `SoDongTacDong`;
END$$

DELIMITER ;
```

Câu lệnh kiểm chứng trên MySQL

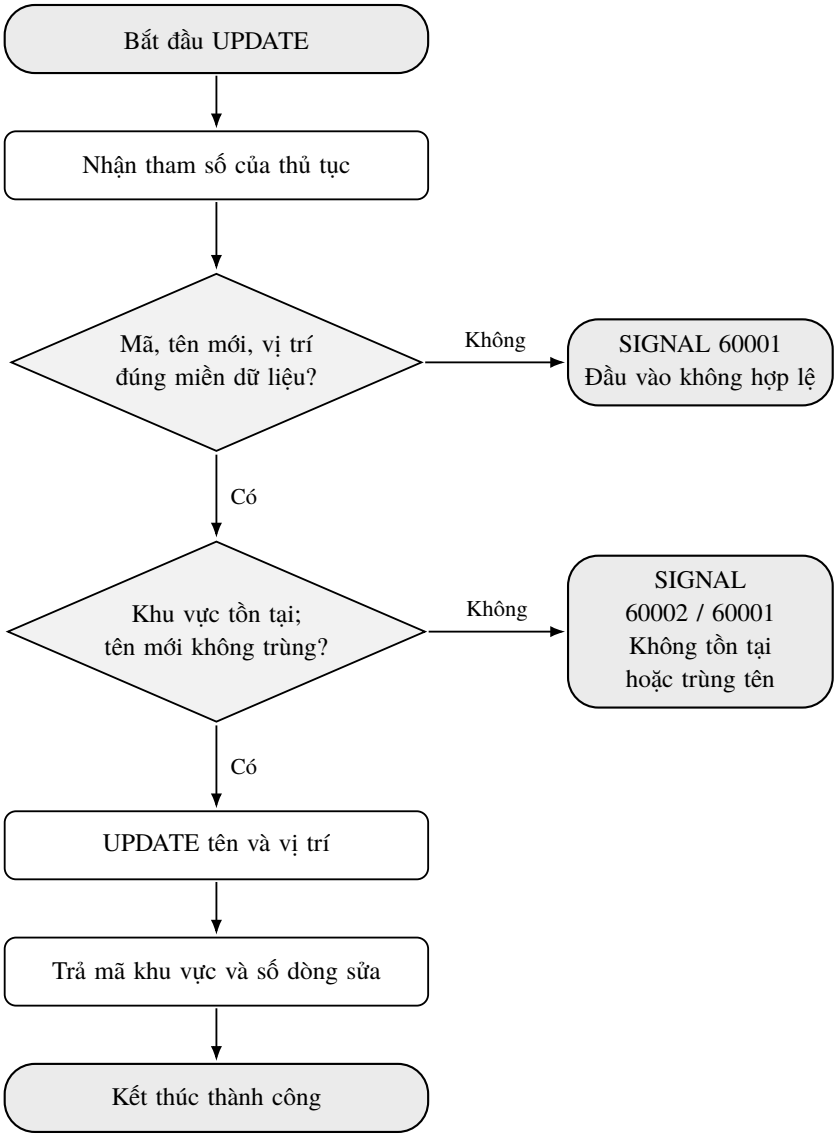
```
CALL sp_khu_vuc_insert('KV99','Khu Demo','Tầng 2');
CALL sp_khu_vuc_insert('KV99','Khu Demo','Tầng 2');
SELECT * FROM KHU_VUC WHERE MaKhuVuc='KV99';
```



Hình 2.2. Kết quả từ DBMS minh họa kiểm tra thủ tục INSERT (trường hợp báo lỗi và trường hợp đúng)

2.1.3 Thủ tục UPDATE: Cập nhật khu vực (sp_khu_vuc_update)

Mô tả: Cập nhật thông tin tên và vị trí khu vực. Kiểm tra khu vực tồn tại, tên mới không trùng khu vực khác. Input: `p_ma_khu_vuc`, `p_ten_moi`, `p_mo_ta_moi`. Output: `ma_khu_vuc`, `thao_tac`, `so_dong_tac_dong`.



Hình 2.3. Lưu đồ minh họa thao tác xử lý thủ tục UPDATE

Câu lệnh tạo thủ tục UPDATE

```
DELIMITER $$

CREATE PROCEDURE `sp_khu_vuc_update`(
    IN pMaKhuVuc VARCHAR(50),
    IN pTenKhuVucMoi VARCHAR(100),
    IN pMoTaViTriMoi VARCHAR(255)
)
proc_label: BEGIN
    DECLARE vMa VARCHAR(10);
    DECLARE vTen VARCHAR(60);

    -- 1. Kiểm tra tham số đầu vào
    IF pMaKhuVuc IS NULL OR TRIM(pMaKhuVuc) = '' OR CHAR_LENGTH(TRIM(pMaKhuVuc)) > 10
    ↪  THEN
        SIGNAL SQLSTATE '45000' SET MYSQL_ERRNO = 60001, MESSAGE_TEXT = 'INVALID_INPUT:
        ↪  MaKhuVuc khong hop le';
    END IF;
```

```
    IF pTenKhuVucMoi IS NULL OR TRIM(pTenKhuVucMoi) = '' OR CHAR_LENGTH(TRIM(pTenKhuVucMoi)) > 60 THEN
        SIGNAL SQLSTATE '45000' SET MYSQL_ERRNO = 60001, MESSAGE_TEXT = 'INVALID_INPUT: TenKhuVuc khong hop le';
    END IF;

    SET vMa = TRIM(pMaKhuVuc);
    SET vTen = TRIM(pTenKhuVucMoi);

    -- 2. Kiểm tra tồn tại
    IF NOT EXISTS (SELECT 1 FROM `KHU_VUC` WHERE `MaKhuVuc` = vMa) THEN
        SIGNAL SQLSTATE '45000' SET MYSQL_ERRNO = 60002, MESSAGE_TEXT = 'NOT_FOUND: Khu vuc khong ton tai';
    END IF;

    -- 3. Kiểm tra trùng lặp tên với khu vực khác
    IF EXISTS (SELECT 1 FROM `KHU_VUC` WHERE `TenKhuVuc` = vTen AND `MaKhuVuc` <> vMa) THEN
        SIGNAL SQLSTATE '45000' SET MYSQL_ERRNO = 60001, MESSAGE_TEXT = 'INVALID_INPUT: TenKhuVuc da ton tai o khu vuc khac';
    END IF;

    -- 4. Cập nhật dữ liệu (không sửa mã khu vực)
    UPDATE `KHU_VUC`
    SET `TenKhuVuc` = vTen,
        `MoTaViTri` = pMoTaViTriMoi
    WHERE `MaKhuVuc` = vMa;

    -- 5. Trả kết quả thực thi
    SELECT vMa AS `MaKhuVuc`, 'UPDATE' AS `ThaoTac`, ROW_COUNT() AS `SoDongTacDong`;
END$$

DELIMITER ;
```

Câu lệnh kiểm chứng trên MySQL

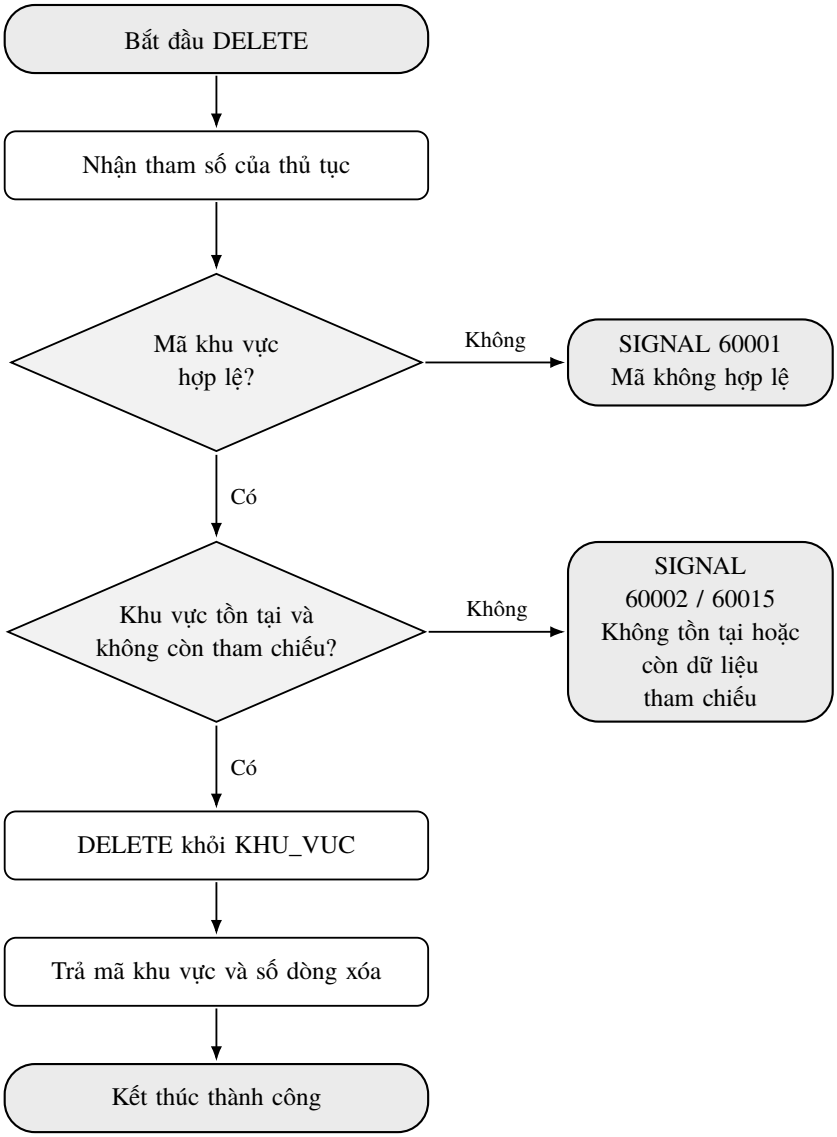
```
CALL sp_khu_vuc_update('KV99','Khu Demo Mới','Tầng 3');
CALL sp_khu_vuc_update('KV_NONE','Không có','');
SELECT * FROM KHU_VUC WHERE MaKhuVuc='KV99';
```



Hình 2.4. Kết quả từ DBMS minh họa kiểm tra thủ tục UPDATE (trường hợp báo lỗi và trường hợp đúng)

2.1.4 Thủ tục DELETE: Xóa khu vực (sp_khu_vuc_delete)

Mô tả: Xóa khu vực không còn hoạt động. Kiểm tra khu vực tồn tại và không còn máy trạm hoặc lịch trực tham chiếu. Input: `p_ma_khu_vuc`. Output: `ma_khu_vuc`, `thao_tac`, `so_dong_tac_dong`.



Hình 2.5. Lưu đồ minh họa thao tác xử lý thủ tục DELETE

Câu lệnh tạo thủ tục DELETE

```
DELIMITER $$

CREATE PROCEDURE `sp_khu_vuc_delete`(
    IN pMaKhuVuc VARCHAR(50)
)
proc_label: BEGIN
    DECLARE vMa VARCHAR(10);

    -- 1. Kiểm tra tham số đầu vào
    IF pMaKhuVuc IS NULL OR TRIM(pMaKhuVuc) = '' OR CHAR_LENGTH(TRIM(pMaKhuVuc)) > 10
    ↪  THEN
        SIGNAL SQLSTATE '45000' SET MYSQL_ERRNO = 60001, MESSAGE_TEXT = 'INVALID_INPUT:
        ↪  MaKhuVuc khong hop le';
    END IF;

    SET vMa = TRIM(pMaKhuVuc);

    -- 2. Kiểm tra tồn tại
```

```
    IF NOT EXISTS (SELECT 1 FROM `KHU_VUC` WHERE `MaKhuVuc` = vMa) THEN
        SIGNAL SQLSTATE '45000' SET MYSQL_ERRNO = 60002, MESSAGE_TEXT = 'NOT_FOUND: Khu vuc khong ton tai';
    END IF;

    -- 3. Kiểm tra khóa ngoại tham chiếu từ MAY_TRAM
    IF EXISTS (SELECT 1 FROM `MAY_TRAM` WHERE `MaKhuVuc` = vMa) THEN
        SIGNAL SQLSTATE '45000' SET MYSQL_ERRNO = 60015, MESSAGE_TEXT = 'DEPENDENCY_CONFLICT: Con may tram thuoc khu vuc nay';
    END IF;

    -- 4. Kiểm tra khóa ngoại tham chiếu từ PHAN_CONG_TRUC
    IF EXISTS (SELECT 1 FROM `PHAN_CONG_TRUC` WHERE `MaKhuVuc` = vMa) THEN
        SIGNAL SQLSTATE '45000' SET MYSQL_ERRNO = 60015, MESSAGE_TEXT = 'DEPENDENCY_CONFLICT: Con phan cong truc thuoc khu vuc nay';
    END IF;

    -- 5. Thực hiện xóa
    DELETE FROM `KHU_VUC` WHERE `MaKhuVuc` = vMa;

    -- 6. Trả kết quả thực thi
    SELECT vMa AS `MaKhuVuc`, 'DELETE' AS `ThaoTac`, ROW_COUNT() AS `SoDongTacDong`;
END$$

DELIMITER ;
```

Câu lệnh kiểm chứng trên MySQL

```
CALL sp_khu_vuc_delete('KV01');
CALL sp_khu_vuc_delete('KV99');
SELECT COUNT(*) AS ConLai FROM KHU_VUC WHERE MaKhuVuc='KV99';
```

MySQL SQL Client

Máy chủ MySQL 8.4.13 · Kết nối local · Phiên kiểm chứng BTL2 L09 Nhóm 10

img_11_crud_delete

```
CALL sp_khu_vuc_delete('KV01');
CALL sp_khu_vuc_delete('KV99');
SELECT COUNT(*) AS ConLai FROM KHU_VUC WHERE MaKhuVuc='KV99';
```

Thực thi SQL btl2_evidence_a62ff5712250 · Hoàn tất

ERROR 60015 (45000): DEPENDENCY_CONFLICT: Con may tram thuoc khu vuc nay

1 dòng · 11.27 ms

MaKhuVuc	ThaoTac	SoDongTacDong
KV99	DELETE	1

1 dòng · 2.39 ms

ConLai
0

SQL được gửi tới MySQL thật. Mỗi ảnh có câu lệnh và đầu ra JSON tương ứng trong evidence/dbms.

Hình 2.6. Kết quả từ DBMS minh họa kiểm tra thủ tục DELETE (trường hợp báo lỗi và trường hợp đúng)

MySQL SQL Client

Máy chủ MySQL 8.4.13 · Kết nối local · Phiên kiểm chứng BTL2 L09 Nhóm 10

routines_created

```
E_NAME,ROUTINE_TYPE FROM information_schema.ROUTINES WHERE ROUTINE_SCHEMA=DATABASE() ORDER BY ROUTINE_TYPE,ROUTINE_NAME;
```

Thực thi SQL btl2_evidence_a62ff5712250 · Hoàn tất

19 dòng · 9.22 ms

ROUTINE_NAME	ROUTINE_TYPE
fn_co_quyen	FUNCTION
fn_han_phien	FUNCTION
fn_so_du_kha_dung	FUNCTION
fn_tien_gio	FUNCTION
fn_tong_chi_vi_rong	FUNCTION
fn_tong_tien_don	FUNCTION
sp_bao_cao_doanh_thu_ngay	PROCEDURE
sp_bao_cao_ton_dich_vu	PROCEDURE
sp_ghi_so_phieu_kho	PROCEDURE
sp_ket_thuc_phien	PROCEDURE
sp_khoa_dong	PROCEDURE
sp_khoa_phien	PROCEDURE
sp_khu_vuc_delete	PROCEDURE
sp_khu_vuc_insert	PROCEDURE
sp_khu_vuc_update	PROCEDURE
sp_mo_phien	PROCEDURE
sp_nap_vi	PROCEDURE
sp_thanh_toan_don	PROCEDURE
sp_tiep_nhan_don	PROCEDURE

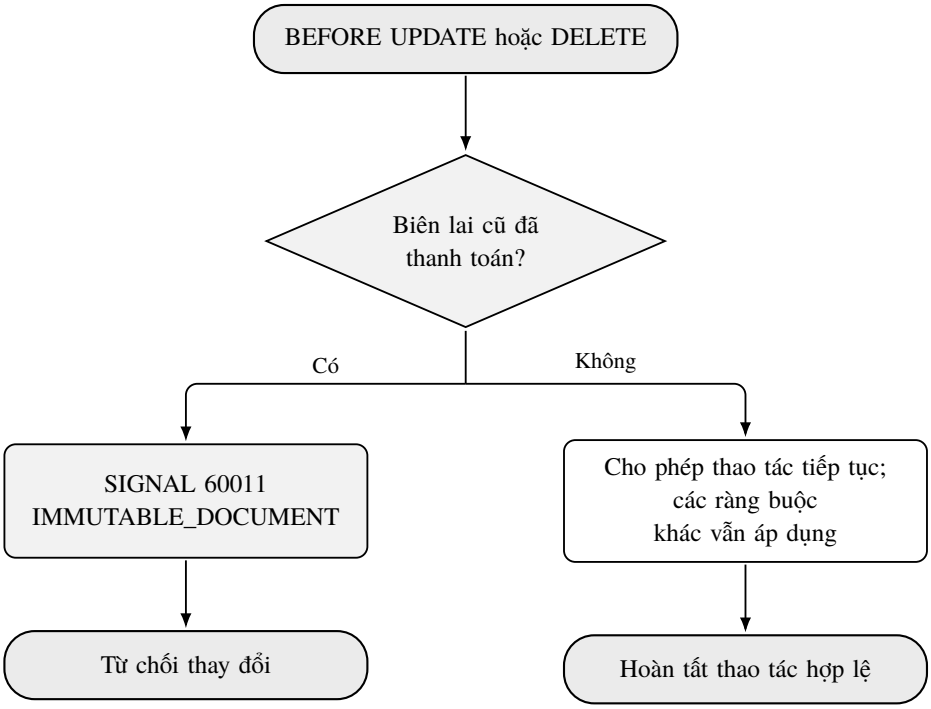
SQL được gửi tới MySQL thật. Mỗi ảnh có câu lệnh và đầu ra JSON tương ứng trong evidence/dbms.

Hình 2.7. Kết quả màn hình hiển thị từ DBMS minh họa cho việc tạo các thủ tục CRUD

2.2 TRIGGER

2.2.1 Trigger 1: Bảo vệ biên lai đã thanh toán (T-P12-01)

Bảng dữ liệu chính liên quan: `BIEN_LAI_TIEN_MAY`. Mô tả: Trigger kiểm tra ràng buộc nghiệp vụ kế toán — không cho phép sửa đổi hoặc xóa biên lai đã ở trạng thái thanh toán (`DA_THANH_TOAN`). Được kích hoạt BEFORE UPDATE và BEFORE DELETE. Vẽ flowchart minh họa xử lý.



Hình 2.8. Lưu đồ minh họa xử lý Trigger 1 bảo vệ biên lai đã thanh toán

Câu lệnh tạo Trigger 1 trên BIEN_LAI_TIEN_MAY

```
1 DELIMITER $$

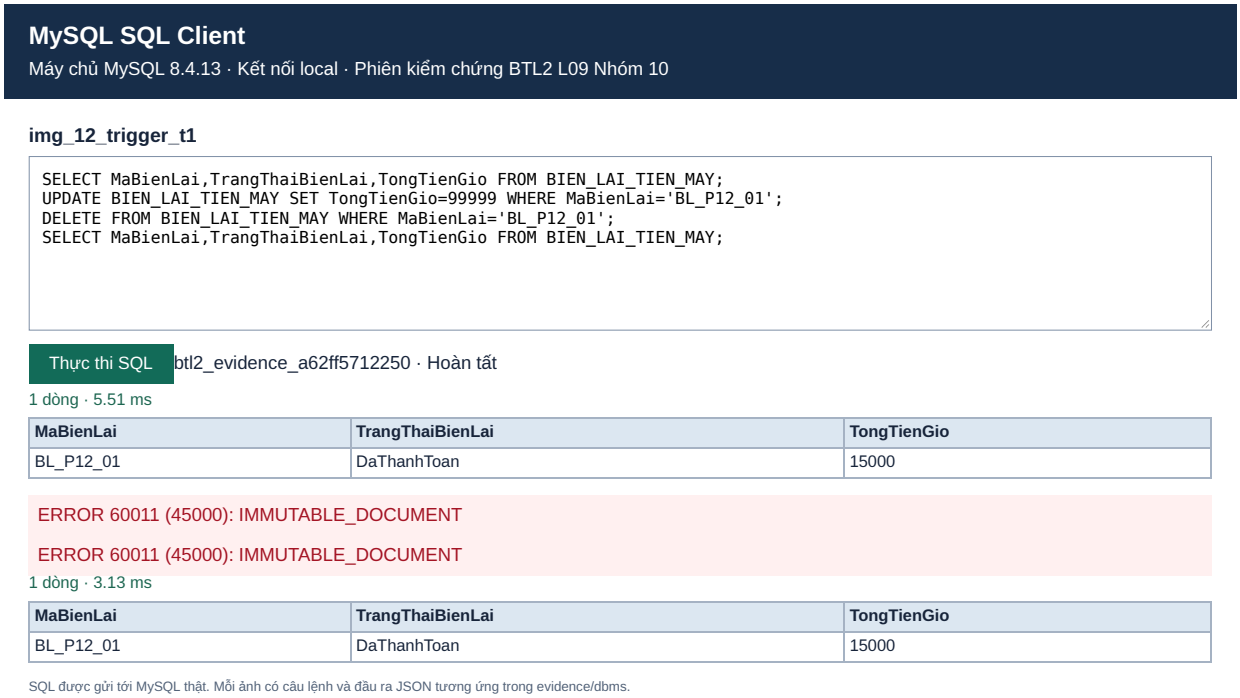
2 CREATE TRIGGER tr_bien_lai_tien_may_update BEFORE UPDATE ON BIEN_LAI_TIEN_MAY FOR EACH
  ↪  ROW BEGIN IF OLD.TrangThaiBienLai='DaThanhToan' THEN SIGNAL SQLSTATE '45000' SET
  ↪  MYSQL_ERRNO=60011,MESSAGE_TEXT='IMMUTABLE_DOCUMENT'; END IF; END$$

3 CREATE TRIGGER tr_bien_lai_tien_may_delete BEFORE DELETE ON BIEN_LAI_TIEN_MAY FOR EACH
  ↪  ROW BEGIN IF OLD.TrangThaiBienLai='DaThanhToan' THEN SIGNAL SQLSTATE '45000' SET
  ↪  MYSQL_ERRNO=60011,MESSAGE_TEXT='IMMUTABLE_DOCUMENT'; END IF; END$$

4 DELIMITER ;
```

Câu lệnh kiểm chứng trên MySQL

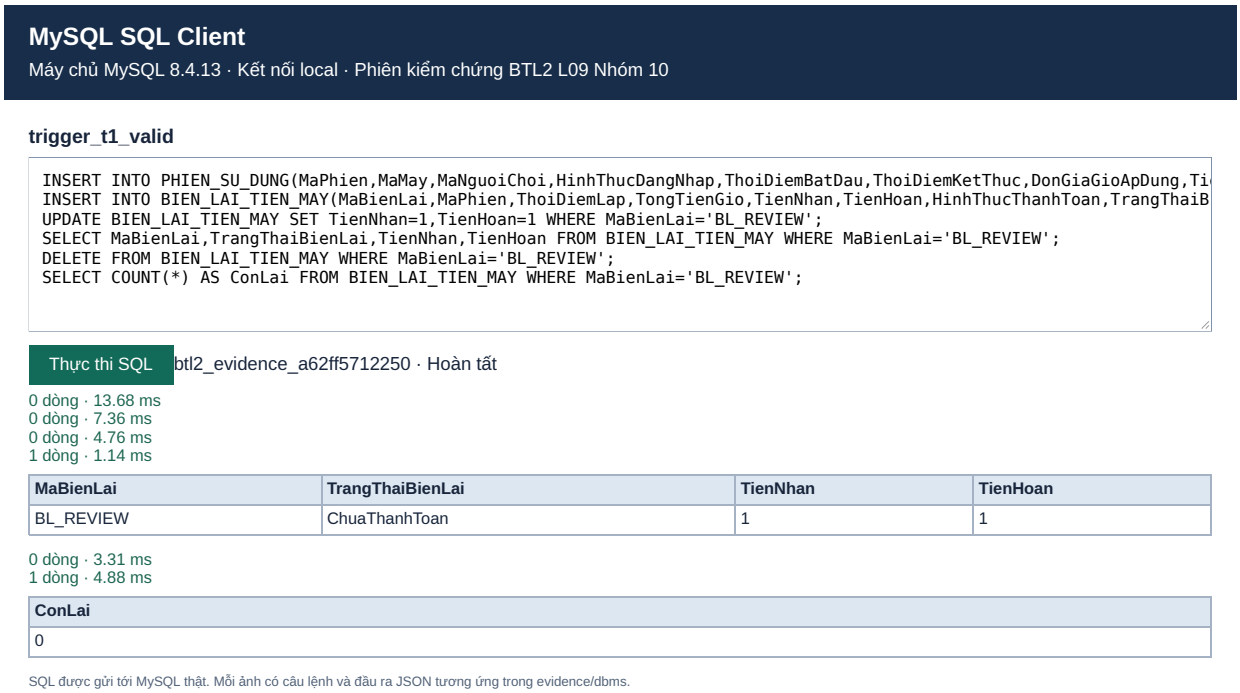
```
SELECT MaBienLai,TrangThaiBienLai,TongTienGio FROM BIEN_LAI_TIEN_MAY;
UPDATE BIEN_LAI_TIEN_MAY SET TongTienGio=99999 WHERE MaBienLai='BL_P12_01';
DELETE FROM BIEN_LAI_TIEN_MAY WHERE MaBienLai='BL_P12_01';
SELECT MaBienLai,TrangThaiBienLai,TongTienGio FROM BIEN_LAI_TIEN_MAY;
```



Hình 2.9. Kết quả DBMS minh họa trường hợp báo lỗi: Chặn sửa và xóa biên lai đã thanh toán

Câu lệnh kiểm chứng trên MySQL

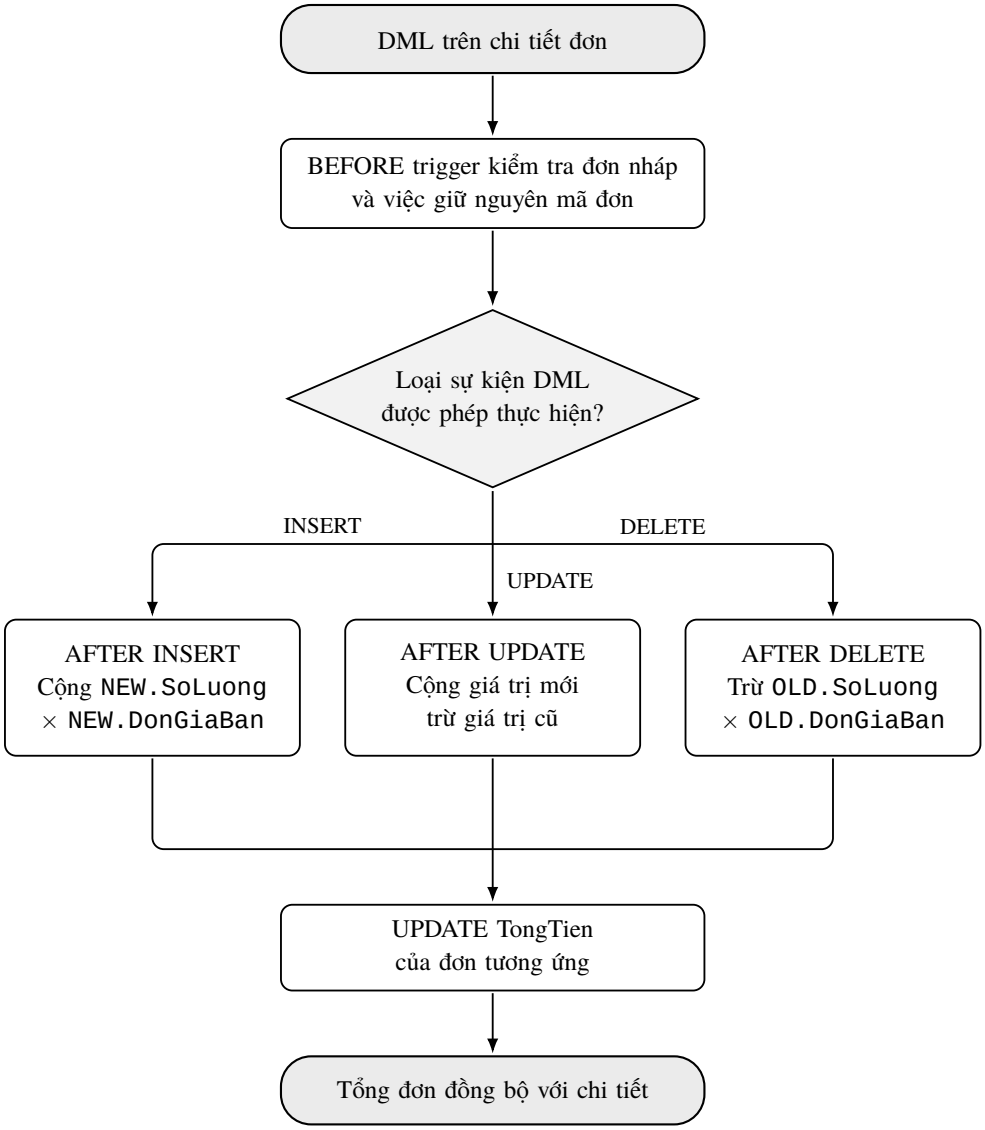
```
INSERT INTO PHIEN_SU_DUNG(MaPhien,MaMay,MaNguoiChoi,HinhThucDangNhap,ThoiDiemBatDau,ThoiDiemKetThuc,DonGiaGioApDung,TienNapTruoc,TienGiuCho,TongTienTamTinh,TrangThaiPhien) VALUES('P_VALID','M04',NULL,'MienPhi','2026-10-23 10:00:00','2026-10-23 11:00:00',0,0,0,0,'DaKetThuc');
INSERT INTO BIEN_LAI_TIEN_MAY(MaBienLai,MaPhien,ThoiDiemLap,TongTienGio,TienNhan,TienHoan,HinhThucThanhToan,TrangThaiBienLai,MaNhanVienThu) VALUES('BL_REVIEW','P_VALID',NOW(),0,0,0,'MienPhi','ChuaThanhToan','TK03');
UPDATE BIEN_LAI_TIEN_MAY SET TienNhan=1,TienHoan=1 WHERE MaBienLai='BL_REVIEW';
SELECT MaBienLai,TrangThaiBienLai,TienNhan,TienHoan FROM BIEN_LAI_TIEN_MAY WHERE MaBienLai='BL_REVIEW';
DELETE FROM BIEN_LAI_TIEN_MAY WHERE MaBienLai='BL_REVIEW';
SELECT COUNT(*) AS ConLai FROM BIEN_LAI_TIEN_MAY WHERE MaBienLai='BL_REVIEW';
```



Hình 2.10. Kết quả DBMS minh họa trường hợp đúng: Cho phép sửa và xóa biên lai chưa thanh toán

2.2.2 Trigger 2: Đồng bộ tổng tiền đơn từ chi tiết (T-P12-02)

Bảng dữ liệu chính liên quan: `CHI_TIET_GOI_DICH_VU`. Bảng dữ liệu phụ liên quan: `DON_GOI_DICH_VU`. Mô tả: Nhóm trigger thao tác ảnh hưởng tới hơn 1 bảng — tự động cập nhật tổng tiền đầu đơn (`tong_tien`) khi thêm, sửa số lượng/giá, hoặc xóa dòng chi tiết món. Kích hoạt AFTER INSERT, AFTER UPDATE, AFTER DELETE.



Hình 2.11. Lưu đồ minh họa xử lý Trigger 2 đồng bộ tổng giá trị đơn hàng

Câu lệnh tạo nhóm Trigger 2 trên CHI_TIET_GOI_DICH_VU

```
DELIMITER $$

CREATE TRIGGER `tr_chi_tiet_goi_dich_vu_after_insert`
AFTER INSERT ON `CHI_TIET_GOI_DICH_VU`
FOR EACH ROW
BEGIN
    UPDATE `DON_GOI_DICH_VU`
    SET `TongTien` = `TongTien` + (NEW.`SoLuong` * NEW.`DonGiaBan`)
    WHERE `MaDonHang` = NEW.`MaDonHang`;
END$$

CREATE TRIGGER `tr_chi_tiet_goi_dich_vu_after_update`
AFTER UPDATE ON `CHI_TIET_GOI_DICH_VU`
FOR EACH ROW
BEGIN
    UPDATE `DON_GOI_DICH_VU`
    SET `TongTien` = `TongTien` + (NEW.`SoLuong` * NEW.`DonGiaBan`) - (OLD.`SoLuong` *
    ↪  OLD.`DonGiaBan`)
```

```
    WHERE `MaDonHang` = NEW.`MaDonHang`;
END$$

CREATE TRIGGER `tr_chi_tiet_goi_dich_vu_after_delete`
AFTER DELETE ON `CHI_TIET_GOI_DICH_VU`
FOR EACH ROW
BEGIN
    UPDATE `DON_GOI_DICH_VU`
    SET `TongTien` = `TongTien` - (OLD.`SoLuong` * OLD.`DonGiaBan`)
    WHERE `MaDonHang` = OLD.`MaDonHang`;
END$$

DELIMITER ;
```

Câu lệnh kiểm chứng trên MySQL

```
INSERT INTO DON_GOI_DICH_VU(MaDonHang,ThoiDiemDat,KenhDat,TongTien,TrangThaiPhucVu,TrangThaiThanhToan,SoTienDaHoan)
VALUES('DH_EVIDENCE',NOW(),'TaiQuay',0,'MoiDat','ChuaThanhToan',0);
SELECT MaDonHang,TongTien FROM DON_GOI_DICH_VU WHERE MaDonHang='DH_EVIDENCE';
```



Hình 2.12. Dữ liệu đơn hàng ban đầu có tổng tiền bằng 0

Câu lệnh kiểm chứng trên MySQL

```
INSERT INTO CHI_TIET_GOI_DICH_VU(MaDonHang,MaDichVu,SoLuong,DonGiaBan)
VALUES('DH_EVIDENCE','DV01',2,15000);
SELECT MaDonHang,TongTien FROM DON_GOI_DICH_VU WHERE MaDonHang='DH_EVIDENCE';
UPDATE CHI_TIET_GOI_DICH_VU SET SoLuong=3 WHERE MaDonHang='DH_EVIDENCE';
SELECT MaDonHang,TongTien FROM DON_GOI_DICH_VU WHERE MaDonHang='DH_EVIDENCE';
DELETE FROM CHI_TIET_GOI_DICH_VU WHERE MaDonHang='DH_EVIDENCE';
SELECT MaDonHang,TongTien FROM DON_GOI_DICH_VU WHERE MaDonHang='DH_EVIDENCE';
```

MySQL SQL Client
Máy chủ MySQL 8.4.13 · Kết nối local · Phiên kiểm chứng BTL2 L09 Nhóm 10

img_13_trigger_t2

```
INSERT INTO CHI_TIET_GOI_DICH_VU(MaDonHang,MaDichVu,SoLuong,DonGiaBan) VALUES('DH_EVIDENCE','DV01',2,15000);
SELECT MaDonHang,TongTien FROM DON_GOI_DICH_VU WHERE MaDonHang='DH_EVIDENCE';
UPDATE CHI_TIET_GOI_DICH_VU SET SoLuong=3 WHERE MaDonHang='DH_EVIDENCE';
SELECT MaDonHang,TongTien FROM DON_GOI_DICH_VU WHERE MaDonHang='DH_EVIDENCE';
DELETE FROM CHI_TIET_GOI_DICH_VU WHERE MaDonHang='DH_EVIDENCE';
SELECT MaDonHang,TongTien FROM DON_GOI_DICH_VU WHERE MaDonHang='DH_EVIDENCE';
```

Thực thi SQL btl2_evidence_a62ff5712250 · Hoàn tất

0 dòng · 13.87 ms
1 dòng · 3.08 ms

MaDonHang	TongTien
DH_EVIDENCE	30000

0 dòng · 6.46 ms
1 dòng · 2.43 ms

MaDonHang	TongTien
DH_EVIDENCE	45000

0 dòng · 10.92 ms
1 dòng · 2.47 ms

MaDonHang	TongTien
DH_EVIDENCE	0

SQL được gửi tới MySQL thật. Mỗi ảnh có câu lệnh và đầu ra JSON tương ứng trong evidence/dbms.

Hình 2.13. Kết quả DBMS minh họa trường hợp đúng: Tổng tiền tự động đồng bộ khi thêm/sửa/xóa dòng món

Câu lệnh kiểm chứng trên MySQL

```
1 INSERT INTO CHI_TIET_GOI_DICH_VU VALUES('DH_P12_01','DV04',1,18000,DEFAULT);
2 UPDATE CHI_TIET_GOI_DICH_VU SET SoLuong=2 WHERE MaDonHang='DH_P12_01';
3 DELETE FROM CHI_TIET_GOI_DICH_VU WHERE MaDonHang='DH_P12_01';
```

MySQL SQL Client
Máy chủ MySQL 8.4.13 · Kết nối local · Phiên kiểm chứng BTL2 L09 Nhóm 10

trigger_t2_errors

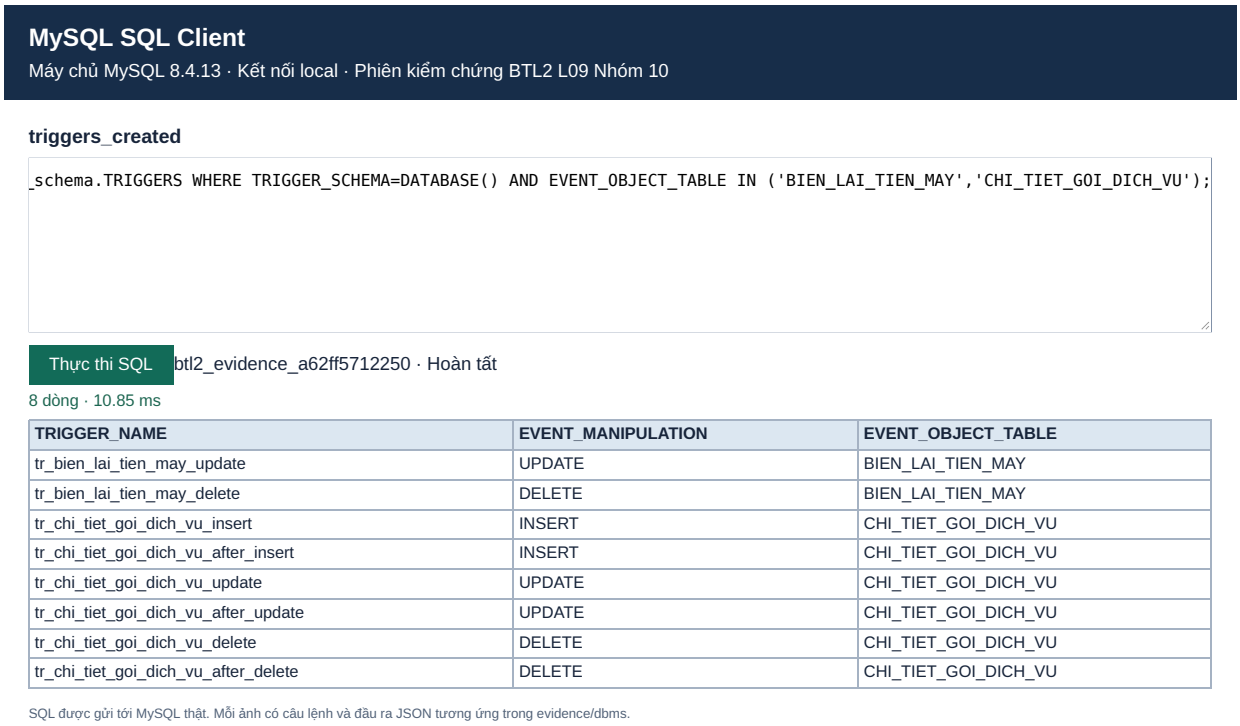
```
INSERT INTO CHI_TIET_GOI_DICH_VU VALUES('DH_P12_01','DV04',1,18000,DEFAULT);
UPDATE CHI_TIET_GOI_DICH_VU SET SoLuong=2 WHERE MaDonHang='DH_P12_01';
DELETE FROM CHI_TIET_GOI_DICH_VU WHERE MaDonHang='DH_P12_01';
```

Thực thi SQL btl2_evidence_a62ff5712250 · Hoàn tất

ERROR 60011 (45000): IMMUTABLE_DOCUMENT
ERROR 60011 (45000): IMMUTABLE_DOCUMENT
ERROR 60011 (45000): IMMUTABLE_DOCUMENT

SQL được gửi tới MySQL thật. Mỗi ảnh có câu lệnh và đầu ra JSON tương ứng trong evidence/dbms.

Hình 2.14. Kết quả DBMS minh họa trường hợp báo lỗi: Chặn thao tác trên đơn đã đóng hoặc đã hủy

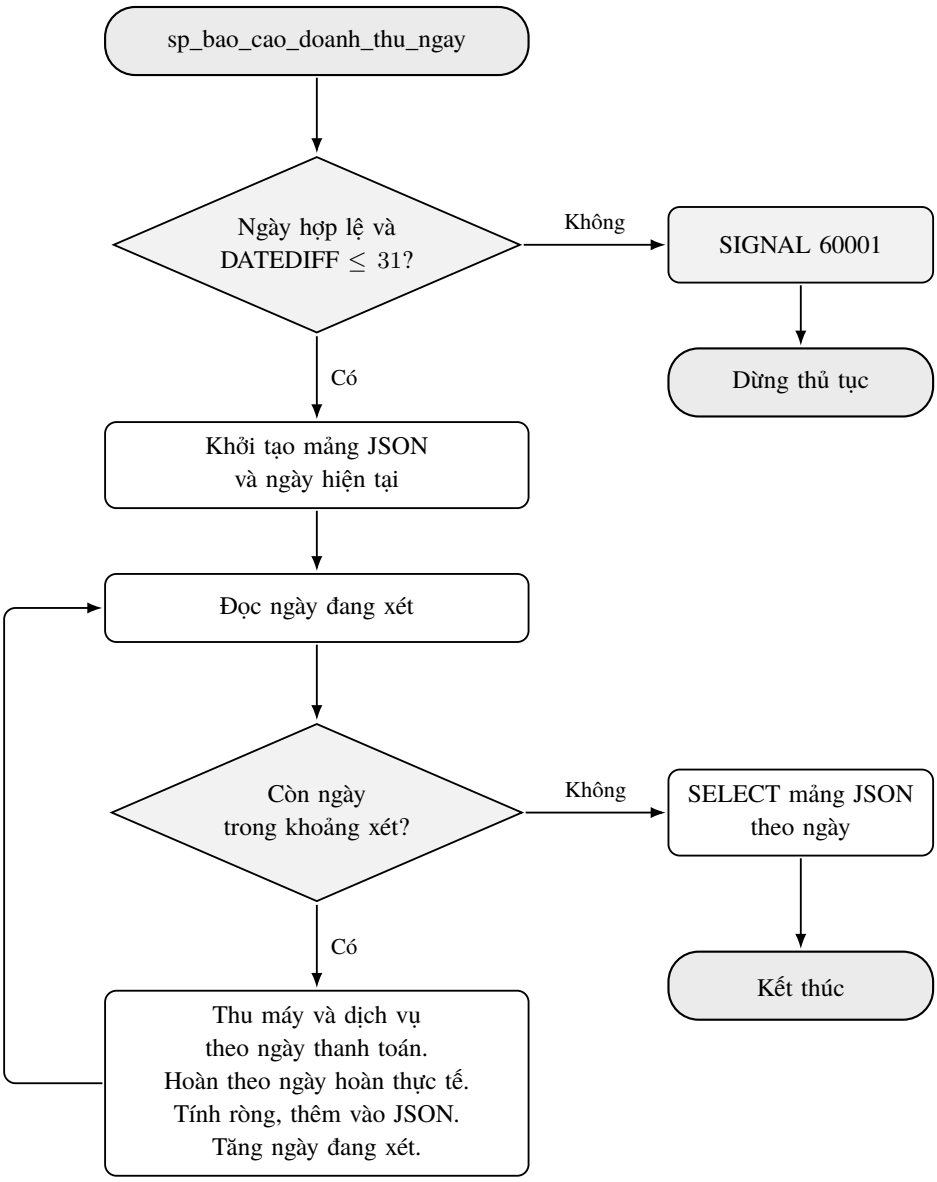


Hình 2.15. Các trigger trong hai yêu cầu đã được tạo

2.3 THỦ TỤC

2.3.1 Thủ tục 1: Báo cáo doanh thu theo ngày (sp_bao_cao_doanh_thu_ngay)

Mô tả thủ tục: Tổng hợp dữ liệu từ nhiều bảng (`BIEN_LAI_TIEN_MAY`, `DON_GOI_DICH_VU`). Sử dụng câu lệnh điều kiện IF kiểm tra ngày hợp lệ và vòng lặp WHILE duyệt qua từng ngày trong khoảng thời gian để tính tổng thu máy, thu dịch vụ và hoàn tiền. Output: Mảng JSON doanh thu chi tiết từng ngày.



Hình 2.16. Lưu đồ minh họa Thủ tục 1 duyệt vòng lặp WHILE tính doanh thu theo ngày

Câu lệnh tạo Thủ tục 1: sp_bao_cao_doanh_thu_ngay

```
DELIMITER $$

CREATE PROCEDURE `sp_bao_cao_doanh_thu_ngay`(
    IN pNgayBatDau DATE,
    IN pNgayKetThuc DATE
)
proc_label: BEGIN
    DECLARE vNgayHienTai DATE;
    DECLARE vThuMay DECIMAL(12,0);
    DECLARE vThuDichVu DECIMAL(12,0);
    DECLARE vHoanDichVu DECIMAL(12,0);
    DECLARE vDanhSach JSON;
    DECLARE vItem JSON;

    -- 1. Kiểm tra tham số đầu vào
    IF pNgayBatDau IS NULL OR pNgayKetThuc IS NULL THEN
```

```
        SIGNAL SQLSTATE '45000' SET MYSQL_ERRNO = 60001, MESSAGE_TEXT = 'INVALID_INPUT: Ngay bat dau va ngay ket thuc khong duoc de trong';
    END IF;

    IF pNgayBatDau > pNgayKetThuc THEN
        SIGNAL SQLSTATE '45000' SET MYSQL_ERRNO = 60001, MESSAGE_TEXT = 'INVALID_INPUT: Ngay bat dau phai nho hon hoac bang ngay ket thuc';
    END IF;

    IF DATEDIFF(pNgayKetThuc, pNgayBatDau) > 31 THEN
        SIGNAL SQLSTATE '45000' SET MYSQL_ERRNO = 60001, MESSAGE_TEXT = 'INVALID_INPUT: Khoang thoi gian bao cao khong vuot qua 31 ngay';
    END IF;

    -- 2. Khởi tạo mảng JSON và biến chạy ngày
    SET vDanhSach = JSON_ARRAY();
    SET vNgayHienTai = pNgayBatDau;

    -- 3. Vòng lặp WHILE duyệt qua từng ngày trong khoảng xét
    WHILE vNgayHienTai <= pNgayKetThuc DO
        -- Thu tiền máy từ các biên lai đã thanh toán trong ngày
        SELECT COALESCE(SUM(`TongTienGio`), 0)
        INTO vThuMay
        FROM `BIEN_LAI_TIEN_MAY`
        WHERE `TrangThaiBienLai` = 'DaThanhToan'
          AND DATE(`ThoiDiemThanhToan`) = vNgayHienTai;

        -- Thu tiền dịch vụ phát sinh theo thời điểm thanh toán
        SELECT COALESCE(SUM(`TongTien`), 0)
        INTO vThuDichVu
        FROM `DON_GOI_DICH_VU`
        WHERE `TrangThaiThanhToan` IN ('DaThanhToan', 'DaHoanTien')
          AND DATE(`ThoiDiemThanhToan`) = vNgayHienTai;

        -- Tiền hoàn dịch vụ phát sinh theo thời điểm hoàn tiền thực tế
        SELECT COALESCE(SUM(`SoTienDaHoan`), 0)
        INTO vHoanDichVu
        FROM `DON_GOI_DICH_VU`
        WHERE `SoTienDaHoan` > 0
          AND `ThoiDiemHoanTien` IS NOT NULL
          AND DATE(`ThoiDiemHoanTien`) = vNgayHienTai;

        -- Đóng gói kết quả ngày thành đối tượng JSON
        SET vItem = JSON_OBJECT(
            'Ngay', DATE_FORMAT(vNgayHienTai, '%Y-%m-%d'),
            'ThuTienMay', CAST(vThuMay AS SIGNED),
            'ThuDichVu', CAST(vThuDichVu AS SIGNED),
            'HoanDichVu', CAST(vHoanDichVu AS SIGNED),
            'DoanhThuRong', CAST((vThuMay + vThuDichVu - vHoanDichVu) AS SIGNED)
        );

        SET vDanhSach = JSON_ARRAY_APPEND(vDanhSach, '$', vItem);

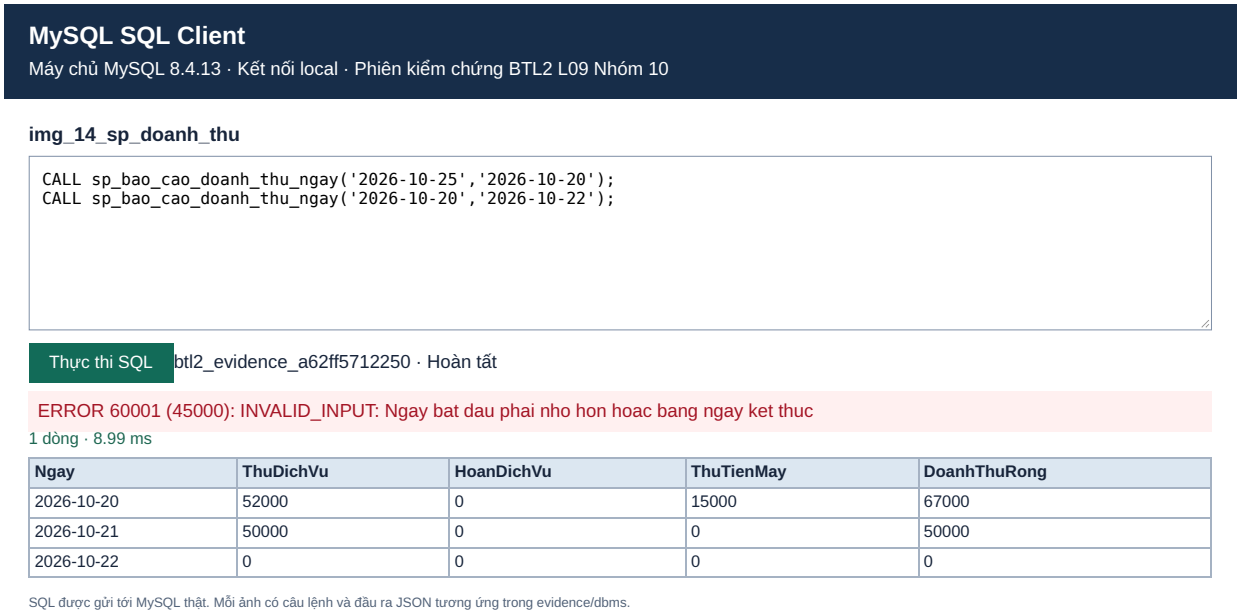
        -- Chuyển sang ngày tiếp theo
        SET vNgayHienTai = DATE_ADD(vNgayHienTai, INTERVAL 1 DAY);
    END WHILE;

    -- 4. Trả một dòng JSON chứa danh sách kết quả từng ngày
    SELECT vDanhSach AS `BaoCaoDoanhThuNgay`;
END$$
```

```
62 DELIMITER ;
```

Câu lệnh kiểm chứng trên MySQL

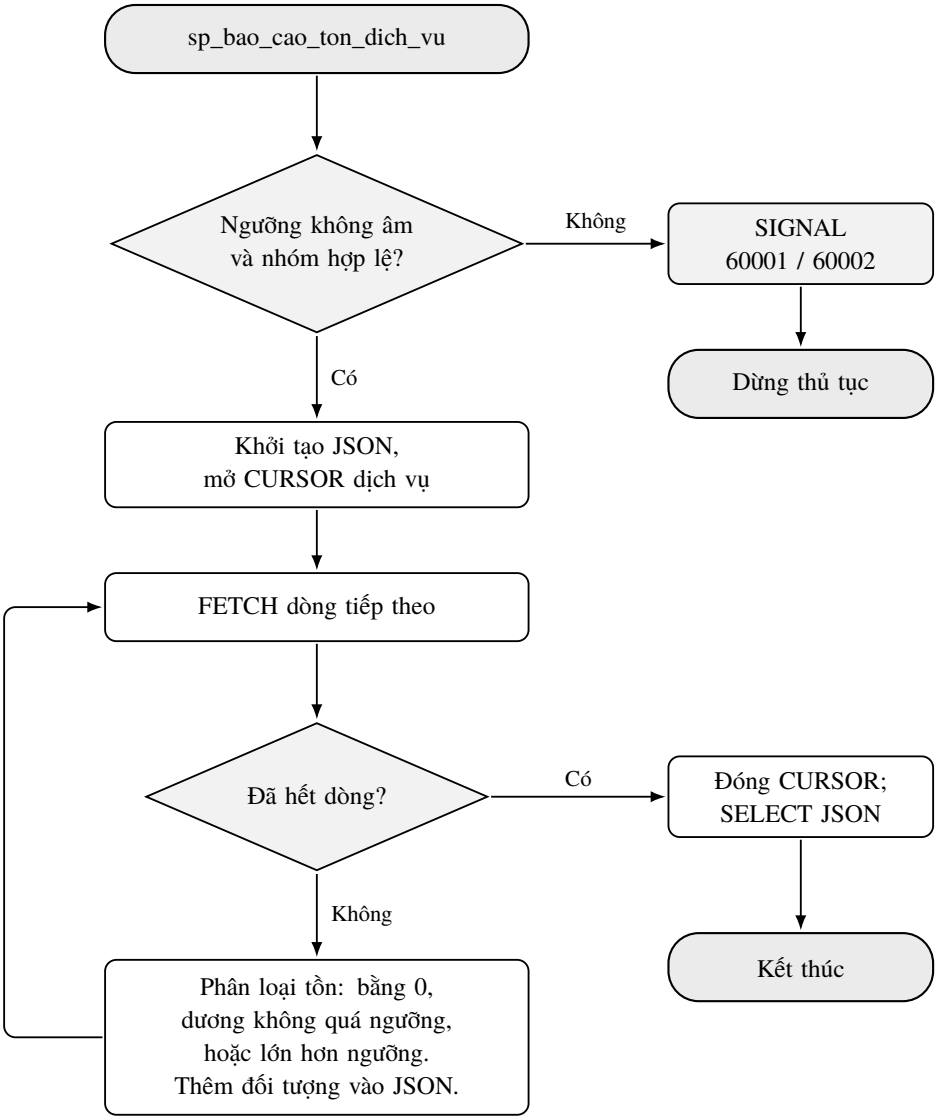
```
1 CALL sp_bao_cao_doanh_thu_ngay('2026-10-25','2026-10-20');
2 CALL sp_bao_cao_doanh_thu_ngay('2026-10-20','2026-10-22');
```



Hình 2.17. Kết quả DBMS minh họa kiểm tra Thủ tục 1 (ca báo lỗi ngày sai và ca đúng trả mảng JSON)

2.3.2 Thủ tục 2: Báo cáo tồn và phân loại dịch vụ (sp_bao_cao_ton_dich_vu)

Mô tả thủ tục: Đọc dữ liệu từ bảng `DICH_VU`, `NHOM_DICH_VU`, `DON_VI_TINH`. Sử dụng CURSOR duyệt danh sách món kết hợp IF/ELSE phân loại trạng thái tồn kho (`HetHang`, `SapHet`, `DuHang`) theo ngưỡng chỉ định. Output: Mảng JSON danh sách dịch vụ và trạng thái phân loại.



Hình 2.18. Lưu đồ minh họa Thủ tục 2 sử dụng CURSOR và IF phân loại tồn kho

Câu lệnh tạo Thủ tục 2: sp_bao_cao_ton_dich_vu

```
DELIMITER $$

CREATE PROCEDURE `sp_bao_cao_ton_dich_vu`(
    IN pMaNhomDV VARCHAR(10),
    IN pNguongTon INT
)
proc_label: BEGIN
    DECLARE vDone INT DEFAULT 0;
    DECLARE vMaDichVu VARCHAR(10);
    DECLARE vTenDichVu VARCHAR(100);
    DECLARE vMaNhom VARCHAR(10);
    DECLARE vTenNhom VARCHAR(60);
    DECLARE vDonViTinh VARCHAR(30);
    DECLARE vGiaBan DECIMAL(12,0);
    DECLARE vSoLuongTon INT;
    DECLARE vTrangThaiKD VARCHAR(20);
    DECLARE vPhanLoai VARCHAR(20);
    DECLARE vDanhSach JSON;
    DECLARE vItem JSON;
```

```
    -- Khai báo CURSOR đọc danh mục dịch vụ theo nhóm hoặc toàn bộ
    DECLARE cur_dv CURSOR FOR
        SELECT d.`MaDichVu`, d.`TenDichVu`, d.`MaNhomDV`, n.`TenNhomDV`,
               v.`TenDonVi`, d.`GiaBanHienHanh`, d.`SoLuongTon`, d.`TrangThaiKinhDoanh`
        FROM `DICH_VU` d
        JOIN `NHOM_DICH_VU` n ON d.`MaNhomDV` = n.`MaNhomDV`
        JOIN `DON_VI_TINH` v ON d.`MaDonVi` = v.`MaDonVi`
        WHERE (pMaNhomDV IS NULL OR TRIM(pMaNhomDV) = '' OR d.`MaNhomDV` =
        ↪  TRIM(pMaNhomDV))
        ORDER BY d.`MaDichVu` ASC;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET vDone = 1;

    -- 1. Kiểm tra tham số đầu vào
    IF pNguongTon IS NULL OR pNguongTon < 0 THEN
        SIGNAL SQLSTATE '45000' SET MYSQL_ERRNO = 60001, MESSAGE_TEXT = 'INVALID_INPUT:
        ↪  Nguong ton phai la so khong am';
    END IF;

    -- 2. Kiểm tra nhóm dịch vụ có tồn tại không nếu được chỉ định
    IF pMaNhomDV IS NOT NULL AND TRIM(pMaNhomDV) <> '' THEN
        IF NOT EXISTS (SELECT 1 FROM `NHOM_DICH_VU` WHERE `MaNhomDV` = TRIM(pMaNhomDV))
        ↪  THEN
            SIGNAL SQLSTATE '45000' SET MYSQL_ERRNO = 60002, MESSAGE_TEXT = 'NOT_FOUND:
            ↪  Nhom dich vu khong ton tai';
        END IF;
    END IF;

    -- 3. Khởi tạo mảng JSON và mở CURSOR
    SET vDanhSach = JSON_ARRAY();
    OPEN cur_dv;

    read_loop: LOOP
        FETCH cur_dv INTO vMaDichVu, vTenDichVu, vMaNhom, vTenNhom,
                          vDonViTinh, vGiaBan, vSoLuongTon, vTrangThaiKD;
        IF vDone = 1 THEN
            LEAVE read_loop;
        END IF;

        -- 4. Sử dụng câu lệnh điều kiện IF phân loại trạng thái tồn kho
        IF vSoLuongTon = 0 THEN
            SET vPhanLoai = 'HetHang';
        ELSEIF vSoLuongTon <= pNguongTon THEN
            SET vPhanLoai = 'SapHet';
        ELSE
            SET vPhanLoai = 'DuHang';
        END IF;

        -- Đóng gói mặt hàng thành đối tượng JSON
        SET vItem = JSON_OBJECT(
            'MaDichVu', vMaDichVu,
            'TenDichVu', vTenDichVu,
            'MaNhomDV', vMaNhom,
            'TenNhomDV', vTenNhom,
            'DonViTinh', vDonViTinh,
            'GiaBanHienHanh', CAST(vGiaBan AS SIGNED),
            'SoLuongTon', vSoLuongTon,
            'TrangThaiKinhDoanh', vTrangThaiKD,
            'PhanLoaiTon', vPhanLoai
        );
```

```
        SET vDanhSach = JSON_ARRAY_APPEND(vDanhSach, '$', vItem);
    END LOOP;

    CLOSE cur_dv;

    -- 5. Trả một dòng JSON chứa danh sách mặt hàng
    SELECT vDanhSach AS `BaoCaoTonDichVu`;
END$$

DELIMITER ;
```

Câu lệnh kiểm chứng trên MySQL

```
CALL sp_bao_cao_ton_dich_vu('NDV02',-1);
CALL sp_bao_cao_ton_dich_vu('NDV02',10);
```

MySQL SQL Client

Máy chủ MySQL 8.4.13 · Kết nối local · Phiên kiểm chứng BTL2 L09 Nhóm 10

img_15_sp_ton_kho

```
CALL sp_bao_cao_ton_dich_vu('NDV02',-1);
CALL sp_bao_cao_ton_dich_vu('NDV02',10);
```

Thực thi SQL btl2_evidence_a62ff5712250 · Hoàn tất

ERROR 60001 (45000): INVALID_INPUT: Nguong ton phai la so khong am

1 dòng · 5.17 ms

MaDichVu	MaNhomDV	DonViTinh	TenDichVu	TenNhomDV	SoLuongTon	PhanLoaiTon	GiaBanHienHanh	TrangThaiKinhDoanh
DV02	NDV02	Chai	Nước ngọt	Đồ uống	100	DuHang	12000	DangKinhDoanh
DV04	NDV02	Chai	Bò húc lon	Đồ uống	8	SapHet	18000	DangKinhDoanh
DV05	NDV02	Chai	Cà phê sữa	Đồ uống	0	HetHang	15000	DangKinhDoanh
DV06	NDV02	Chai	Trà chanh sả	Đồ uống	45	DuHang	10000	NgungKinhDoanh

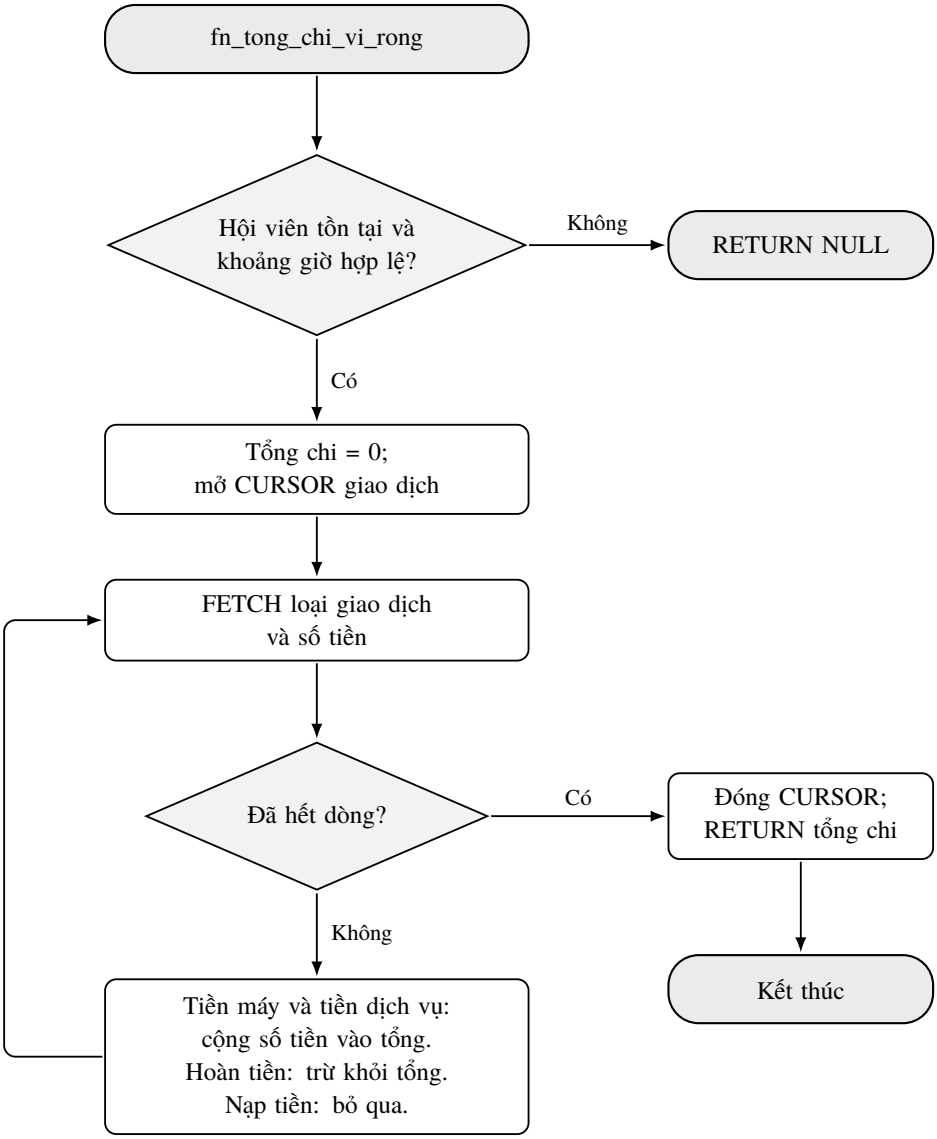
SQL được gửi tới MySQL thật. Mỗi ảnh có câu lệnh và đầu ra JSON tương ứng trong evidence/dbms.

Hình 2.19. Kết quả DBMS minh họa kiểm tra Thủ tục 2 (ca báo lỗi ngưỡng âm và ca đúng trả dữ liệu)

2.4 HÀM

2.4.1 Hàm 1: Tổng chi ví ròng (fn_tong_chi_vi_rong)

Mô tả hàm: Đọc dữ liệu từ bảng `GIAO_DICH_VI` và `NGUOI_CHOI`. Sử dụng CURSOR duyệt qua lịch sử giao dịch kết hợp câu lệnh IF kiểm tra loại giao dịch (`TruTienMay`, `TruTienDichVu`, `HoanTien`) để tính tổng số tiền chi ròng của hội viên trong một khoảng thời gian. Input: `p_ma_nguoi_choi`, `p_tu_gio`, `p_den_gio`. Output: DECIMAL(18,0).



Hình 2.20. Lưu đồ minh họa Hàm 1 dùng CURSOR và IF tính tổng chi ròng

Câu lệnh tạo Hàm 1: fn_tong_chi_vi_rong

```
DELIMITER $$

CREATE FUNCTION `fn_tong_chi_vi_rong`(
    pMaNguoiChoi VARCHAR(15),
    pBatDau DATETIME,
    pKetThuc DATETIME
) RETURNS DECIMAL(18,0)
READS SQL DATA
BEGIN
    DECLARE vDone INT DEFAULT 0;
    DECLARE vLoaiGD VARCHAR(20);
    DECLARE vSoTien DECIMAL(12,0);
    DECLARE vTongChi DECIMAL(18,0) DEFAULT 0;

    -- Khai báo CURSOR đọc giao dịch ví thành công trong khoảng thời gian [pBatDau,
    ↪  pKetThuc)
    DECLARE cur_gd CURSOR FOR
        SELECT `LoaiGiaoDich`, `SoTien`
```

```
        FROM `GIAO_DICH_VI`
        WHERE `MaNguoiChoi` = pMaNguoiChoi
          AND `TrangThaiGD` = 'ThanhCong'
          AND `ThoiDiemGD` >= pBatDau
          AND `ThoiDiemGD` < pKetThuc;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET vDone = 1;

    -- 1. Phân biệt đối tượng không tồn tại (trả về NULL)
    IF NOT EXISTS (SELECT 1 FROM `NGUOI_CHOI` WHERE `MaTaiKhoan` = pMaNguoiChoi) THEN
        RETURN NULL;
    END IF;

    -- 2. Kiểm tra khoảng thời gian hợp lệ
    IF pBatDau IS NULL OR pKetThuc IS NULL OR pBatDau >= pKetThuc THEN
        RETURN NULL;
    END IF;

    -- 3. Mở CURSOR và duyệt qua các giao dịch
    OPEN cur_gd;

    read_loop: LOOP
        FETCH cur_gd INTO vLoaiGD, vSoTien;
        IF vDone = 1 THEN
            LEAVE read_loop;
        END IF;

        -- Điều kiện xử lý loại giao dịch:
        -- Cộng phí dịch vụ và tiền máy qua ví, trừ các khoản hoàn tiền, bỏ qua nạp tiền
        IF vLoaiGD IN ('TruTienMay', 'TruTienDichVu') THEN
            SET vTongChi = vTongChi + vSoTien;
        ELSEIF vLoaiGD = 'HoanTien' THEN
            SET vTongChi = vTongChi - vSoTien;
        END IF;
    END LOOP;

    CLOSE cur_gd;

    RETURN vTongChi;
END$$

DELIMITER ;
```

Câu lệnh kiểm chứng trên MySQL

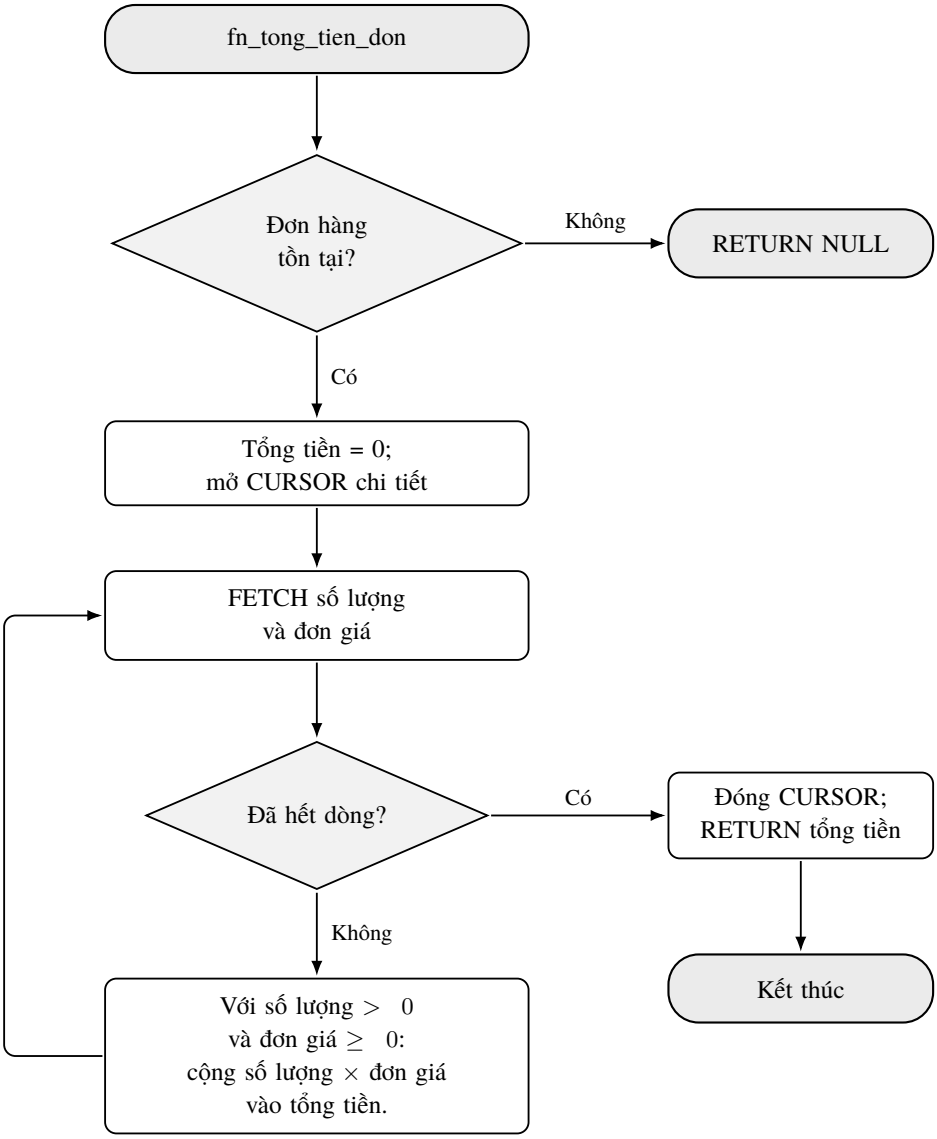
```
SELECT fn_tong_chi_vi_rong('TK02','2026-10-01','2026-11-01') AS TongChi;
SELECT fn_tong_chi_vi_rong('TK_NONE','2026-10-01','2026-11-01') AS KhongTonTai;
```



Hình 2.21. Kết quả DBMS minh họa kiểm tra Hàm 1 (ca đúng trả tổng tiền và ca sai mã trả NULL)

2.4.2 Hàm 2: Tổng tiền từ dòng chi tiết đơn (fn_tong_tien_don)

Mô tả hàm: Đọc dữ liệu từ bảng `CHI_TIET_GOI_DICH_VU`. Sử dụng CURSOR duyệt từng dòng chi tiết của đơn hàng kết hợp kiểm tra IF điều kiện số lượng và đơn giá hợp lệ, sau đó cộng dồn thành tiền của toàn bộ đơn. Input: `p_ma_don_goi`. Output: DECIMAL(18,0).



Hình 2.22. Lưu đồ minh họa Hàm 2 dùng CURSOR tính tổng tiền đơn hàng

Câu lệnh tạo Hàm 2: fn_tong_tien_don

```
DELIMITER $$

CREATE FUNCTION `fn_tong_tien_don`(
    pMaDonHang VARCHAR(20)
) RETURNS DECIMAL(18,0)
READS SQL DATA
BEGIN
    DECLARE vDone INT DEFAULT 0;
    DECLARE vSoLuong INT;
    DECLARE vDonGiaBan DECIMAL(12,0);
    DECLARE vTongTien DECIMAL(18,0) DEFAULT 0;

    -- Khai báo CURSOR duyệt các dòng chi tiết của đơn hàng
    DECLARE cur_ct CURSOR FOR
        SELECT `SoLuong`, `DonGiaBan`
        FROM `CHI_TIET_GOI_DICH_VU`
        WHERE `MaDonHang` = pMaDonHang;
```

```
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET vDone = 1;

    -- 1. Kiểm tra tồn tại đơn hàng (trả NULL nếu không tồn tại)
    IF NOT EXISTS (SELECT 1 FROM `DON_GOI_DICH_VU` WHERE `MaDonHang` = pMaDonHang) THEN
        RETURN NULL;
    END IF;

    -- 2. Mở CURSOR và cộng dồn qua từng dòng
    OPEN cur_ct;

    read_loop: LOOP
        FETCH cur_ct INTO vSoLuong, vDonGiaBan;
        IF vDone = 1 THEN
            LEAVE read_loop;
        END IF;

        IF vSoLuong > 0 AND vDonGiaBan >= 0 THEN
            SET vTongTien = vTongTien + (vSoLuong * vDonGiaBan);
        END IF;
    END LOOP;

    CLOSE cur_ct;

    RETURN vTongTien;
END$$

DELIMITER ;
```

Câu lệnh kiểm chứng trên MySQL

```
SELECT fn_tong_tien_don('DH_P12_01') AS TongTien;
SELECT fn_tong_tien_don('DH_NONE') AS KhongTonTai;
```

MySQL SQL Client

Máy chủ MySQL 8.4.13 · Kết nối local · Phiên kiểm chứng BTL2 L09 Nhóm 10

img_17_fn_tien_don

```
SELECT fn_tong_tien_don('DH_P12_01') AS TongTien;
SELECT fn_tong_tien_don('DH_NONE') AS KhongTonTai;
```

Thực thi SQL btl2_evidence_a62ff5712250 · Hoàn tất

1 dòng · 6.12 ms

TongTien
52000

1 dòng · 2.87 ms

KhongTonTai
NULL

SQL được gửi tới MySQL thật. Mỗi ảnh có câu lệnh và đầu ra JSON tương ứng trong evidence/dbms.

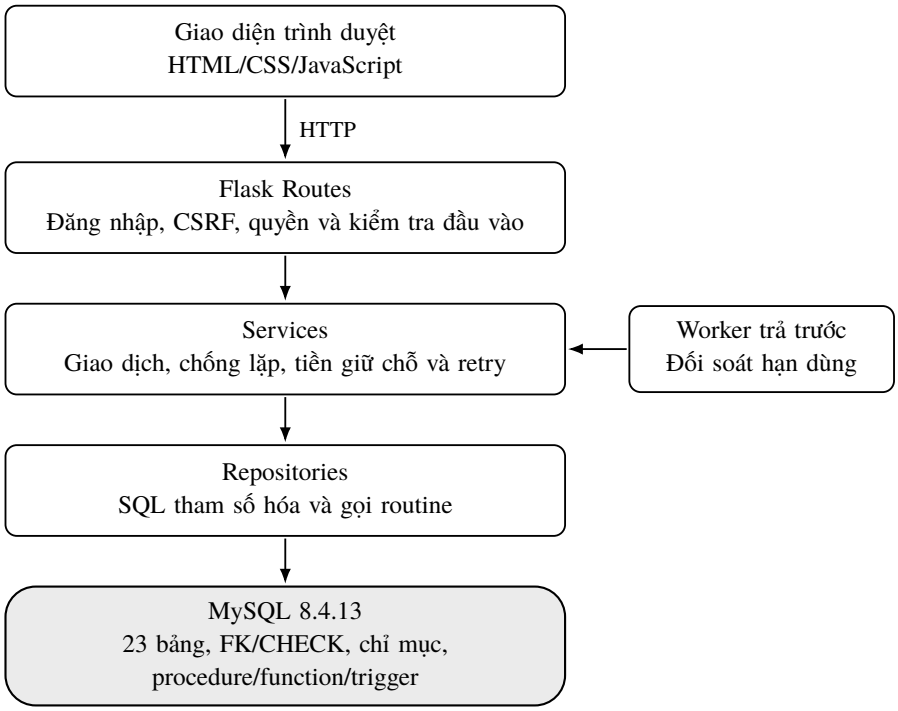
Hình 2.23. Kết quả DBMS minh họa kiểm tra Hàm 2 (ca đúng trả tổng tiền và ca sai mã trả NULL)

PHẦN 3

PHẦN THÊM: Kiểm chứng và tích hợp ứng dụng

3.1 Kiến trúc và giao dịch

Giao diện HTML/CSS/JavaScript gọi REST API Flask trên cùng origin. Backend tách tuyến HTTP, dịch vụ nghiệp vụ và lớp truy cập dữ liệu. Dịch vụ nắm ranh giới BEGIN/COMMIT/ROLLBACK trên một kết nối; routine được gọi trong giao dịch đó, không tự chốt độc lập. Runtime dùng tài khoản hạn quyền, không cấp trực tiếp quyền cập nhật số dư ví hoặc tồn kho.



Hình 3.1. Luồng giao tiếp giữa giao diện, backend và MySQL

Xác thực dùng phiên đăng nhập và kiểm tra CSRF cho API ghi. Dữ liệu đầu vào bị kiểm tra miền kiểu và phạm vi; mã yêu cầu được dùng để phân biệt gửi lặp với xung đột nội dung. Khi có tranh chấp, dữ liệu được đọc lại dưới khóa thay vì tin vào trạng thái đã đọc trước giao dịch.

3.2 Ba luồng kiểm chứng

1. **Phiên chơi:** chọn máy, mở phiên, kết thúc, đối chiếu biên lai và giải phóng máy. Với trả trước, giờ sử dụng và số dư khả dụng được kiểm soát tại server.
2. **Ví và dịch vụ:** nạp ví, gửi lại cùng mã, tạo/tiếp nhận/thanh toán đơn, đối chiếu số dư và tồn. Thanh toán đơn của phiên cũ vẫn phải tính tiền giữ chỗ của phiên đang hoạt động.
3. **Kho và báo cáo:** phiếu nháp không đổi tồn; ghi sổ chỉ tác động một lần; xem báo cáo với khoảng ngày chứa dữ liệu phát sinh.

3.3 Tổng hợp bộ kiểm thử

Bảng 3.1. Kết quả kiểm chứng được lưu trong hồ sơ nghiệm thu

Nhóm ca	PASS
API: quyền, đầu vào, chống lặp	30
Integration: rollback, retry, khóa và tranh chấp	45
SQL P1–2	14
E2E HTTP và Chrome ba luồng	4
Ba ca hồi quy bổ sung	3
Tổng; không FAIL, ERROR hoặc SKIP	**96**

Đây là kết quả của bộ ca đã triển khai, không phải khẳng định mọi trạng thái có thể có đều được bao phủ. Dữ liệu test và tài khoản test được tạo riêng rồi dọn sau từng ca; không reset database ứng dụng. JUnit và log gốc nằm trong `evidence/final_verification/`.

3.3.1 Các ca biên có ảnh hưởng trực tiếp đến tính đúng

- Đơn đặt, thanh toán và hoàn ở ba ngày khác nhau được phân bổ đúng mốc thu/hoàn.
- Ví 100.000đ đang giữ 90.000đ cho phiên mới không được trả đơn cũ 15.000đ.
- Đơn tổng 10 tỷ và tên dịch vụ 90 ký tự không bị tràn/cắt trong hai routine liên quan.
- Gửi lại phiếu kho khác loại trả 409; page quá lớn trả 422, không thành lỗi 500.

3.4 Cài mới và đối chiếu từ gói bàn giao

Bộ cài từ bản ZIP đã được chạy trên database/user mới: tạo 23 bảng, nạp hai đơn seed P1–2, mở giao diện và đăng nhập thành công. Chạy seed lại giữ nguyên dữ liệu; bộ seed một phần bị từ chối để tránh ghi đè không rõ trạng thái.

Các bước vận hành: cài thư viện đã khóa phiên bản, chạy script demo để sinh cấu hình riêng và khởi động MySQL/Flask, sau đó sử dụng tài khoản demo. Mật khẩu quản trị không được đưa vào báo cáo hay ZIP. Hướng dẫn đầy đủ ở README và runbook; kết quả phép thử từ ZIP ở `evidence/final_verification/zip_smoke.json`.

3.5 Phạm vi chưa đưa vào demo

Ứng dụng hiện minh họa ba luồng trên, chưa khóa máy vật lý. API chưa bao gồm đầy đủ hủy/hoàn, quản trị tài khoản, danh mục và ca trực. Các thao tác SQL quản trị ngoài API cần được đối chiếu riêng với toàn bộ quy tắc BTL1. Phần trình bày ứng dụng theo đợt riêng của môn học cần tuân thủ yêu cầu của đợt đó; báo cáo này tập trung vào Phần 1–2.

PHẦN A

BẢNG KÊ NHIỆM VỤ VÀ ĐÁNH GIÁ

A.1 Bảng kê nhiệm vụ

Bảng dưới kế thừa phân công đang lưu trong hồ sơ nhóm. Mỗi hàng chỉ đánh dấu một người phụ trách chính; nội dung tích hợp có thể được kiểm tra chéo. Các cột dùng bốn số cuối MSSV để tránh thu hẹp cột mô tả. Tỉ lệ đóng góp là mức đề xuất, không phải điểm đã được giảng viên xác nhận.

Bảng A.1. Danh sách thành viên và tỷ lệ điểm đề xuất

MSSV	Họ và tên	Đề xuất
2410709	Nguyễn Thành Đạt	20%
2410694	Nguyễn Minh Đạt	20%
2410696	Nguyễn Phúc Bảo Đạt	20%
2413095	Nguyễn Hữu Tây	20%
2413181	Trần Tiến Thành	20%

Bảng A.2. Bảng kê người phụ trách từng nội dung

Nội dung	0694	0696	3095	0709	3181
Tạo bảng dữ liệu phần 1: 12 bảng danh tính và máy trạm	x				
Tạo bảng dữ liệu phần 2: 11 bảng dịch vụ, kho, ca trực và ví			x		
Tạo các ràng buộc khóa chính, duy nhất và miền giá trị (CHECK)	x				
Tạo các ràng buộc khóa ngoại bảo toàn toàn vẹn tham chiếu			x		
Demo kiểm tra tạo bảng, ràng buộc và kiểm đếm 23 bảng thành công	x				
Tạo chỉ mục phần 1: ix_dich_vu_filter tăng tốc lọc thực đơn					x
Tạo chỉ mục phần 2 cho các bảng vận hành		x			
Đo lường hiệu năng chỉ mục trước và sau bằng EXPLAIN ANALYZE					x
Câu truy vấn 1 (Q-P12-01) - Bảng liên quan: MAY_TRAM, KHU_VUC	x				
Câu truy vấn 2 (Q-P12-02) - Bảng liên quan: DICH_VU, NHOM_DICH_VU		x			
Câu truy vấn 3 (Q-P12-03) - Bảng liên quan: DON_GOI_DICH_VU		x			
Câu truy vấn 4 (Q-P12-04) - Bảng liên quan: CA_TRUC, PHAN_CONG_TRUC					x
Câu truy vấn 5 (Q-P12-05) - Bảng liên quan: NGUOI_CHOI, GIAO_DICH_VI			x		
Thủ tục INSERT (sp_khu_vuc_insert) - Bảng: KHU_VUC	x				

Nội dung (tiếp)	0694	0696	3095	0709	3181
Thủ tục UPDATE (sp_khu_vuc_update) - Bảng: KHU_VUC	x				
Thủ tục DELETE (sp_khu_vuc_delete) - Bảng: KHU_VUC	x				
Demo kiểm tra ba thủ tục CRUD khu vực với các ca đúng và lỗi	x				
Trigger 1 (T-P12-01) - Bảng: BIEN_LAI_TIEN_MAY			x		
Demo kiểm tra Trigger 1 chặn sửa/xóa biên lai đã thanh toán			x		
Trigger 2 (T-P12-02) - Bảng: CHI_TIET_GOI_DICH_VU, DON_GOI_DICH_VU				x	
Demo kiểm tra Trigger 2 duy trì tổng tiền đầu đơn từ chi tiết				x	
Thủ tục 1 (sp_bao_cao_doanh_thu_ngay) - Bảng: BIEN_LAI, DON_GOI		x			
Demo kiểm tra Thủ tục 1 tổng hợp doanh thu theo ngày		x			
Thủ tục 2 (sp_bao_cao_ton_dich_vu) - Bảng: DICH_VU, NHOM_DICH_VU					x
Demo kiểm tra Thủ tục 2 phân loại tồn kho bằng CURSOR và IF					x
Hàm 1 (fn_tong_chi_vi_rong) - Bảng: GIAO_DICH_VI, NGUOI_CHOI			x		
Demo kiểm tra Hàm 1 tính tổng chi ví ròng			x		
Hàm 2 (fn_tong_tien_don) - Bảng: CHI_TIET_GOI_DICH_VU				x	
Demo kiểm tra Hàm 2 tính tổng tiền đơn hàng				x	
Kiến trúc tích hợp Backend Flask và Test suite tự động 79 ca				x	
Thiết kế sơ đồ giải thuật lưu đồ TikZ cho toàn bộ Routine	x				
Kịch bản dữ liệu và benchmark đo lường chỉ mục		x			

A.2 Đánh giá của giáo viên

(Điền danh sách thành viên và % đóng góp bên dưới và để trống các cột còn lại)

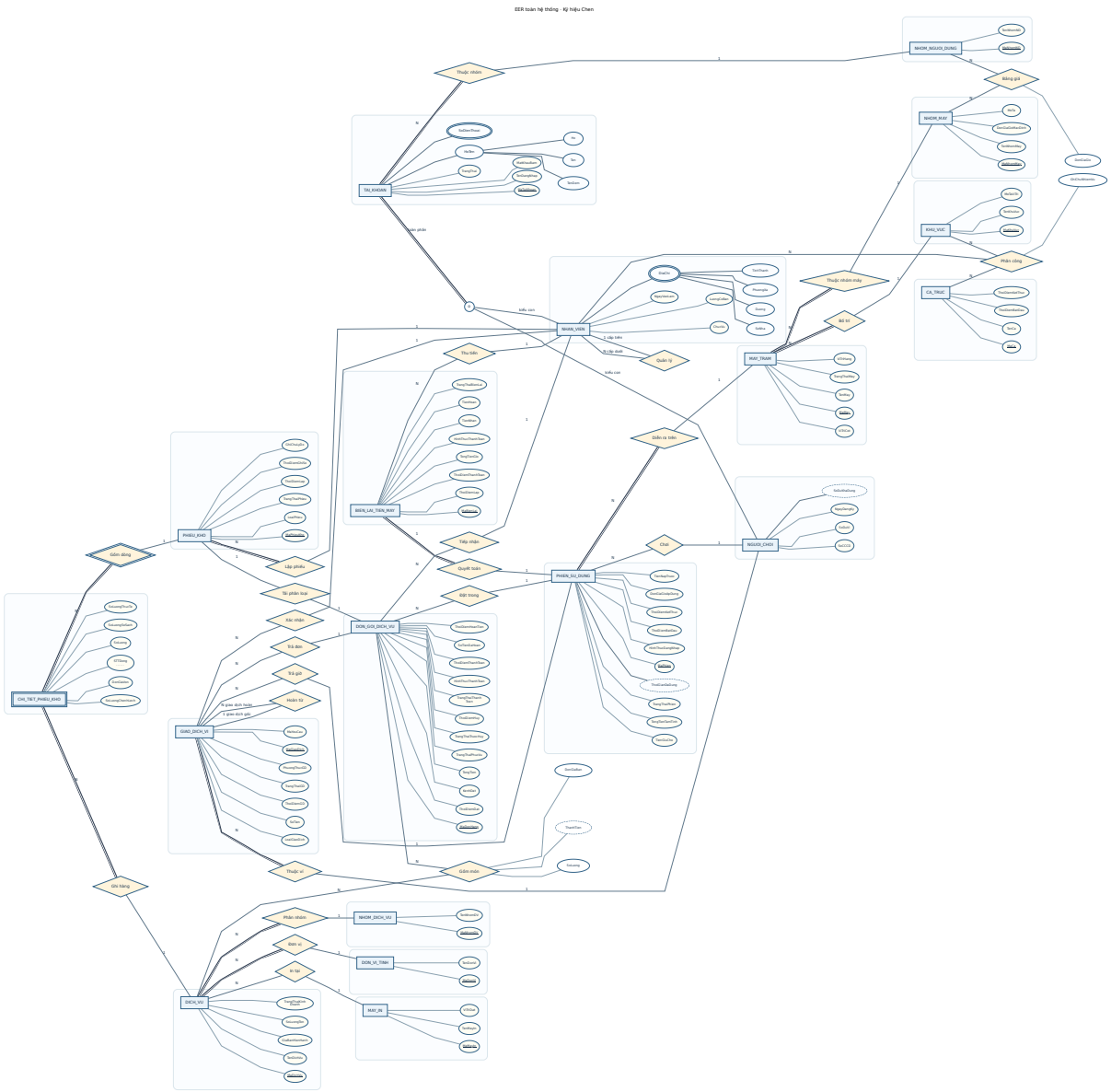
Họ tên	MSSV	Đề xuất	Nhận xét	Điểm trừ/cộng	Tổng
Nguyễn Thành Đạt	2410709	20%			
Nguyễn Minh Đạt	2410694	20%			
Nguyễn Phúc Bảo Đạt	2410696	20%			
Nguyễn Hữu Tây	2413095	20%			
Trần Tiến Thành	2413181	20%			

PHẦN B

PHỤ LỤC: Thiết kế ERD cập nhật mới nhất

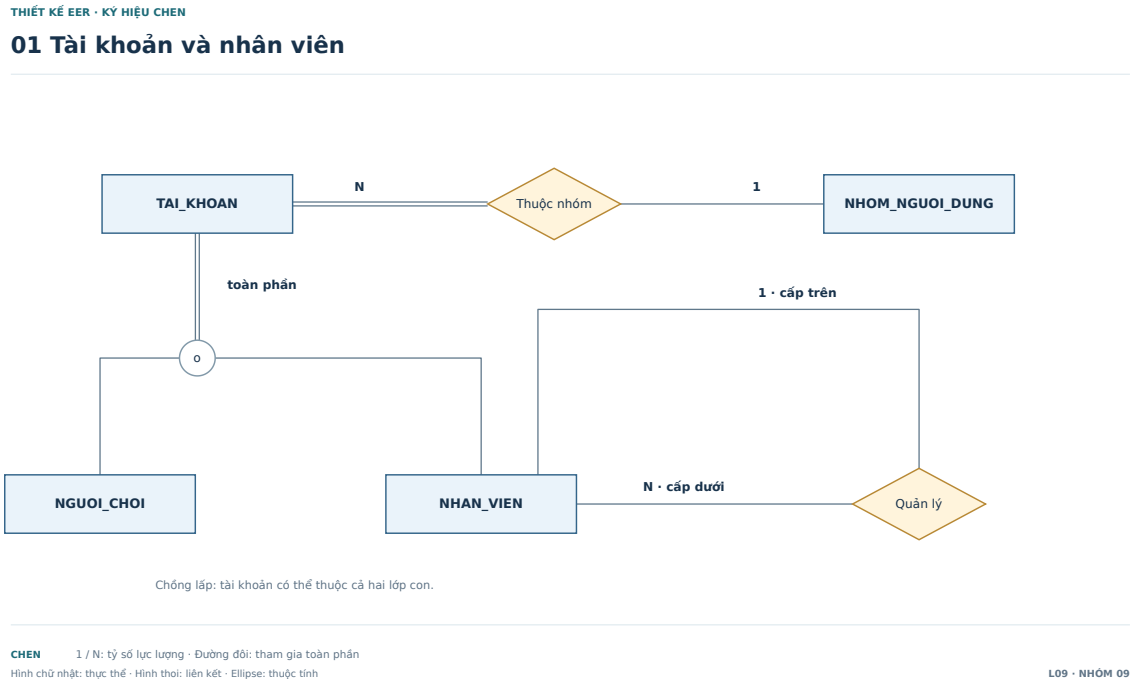
B.1 Sơ đồ tổng thể dạng vector

Sơ đồ giữ ký hiệu EER Chen của BTL1. Bản tổng quan trình bày toàn hệ thống; các trang tiếp theo phóng riêng từng phân hệ. Tệp PDF/SVG nguồn được kèm trong bộ LaTeX, cho phép phóng to mà không vỡ nét hoặc dùng để in khổ lớn.



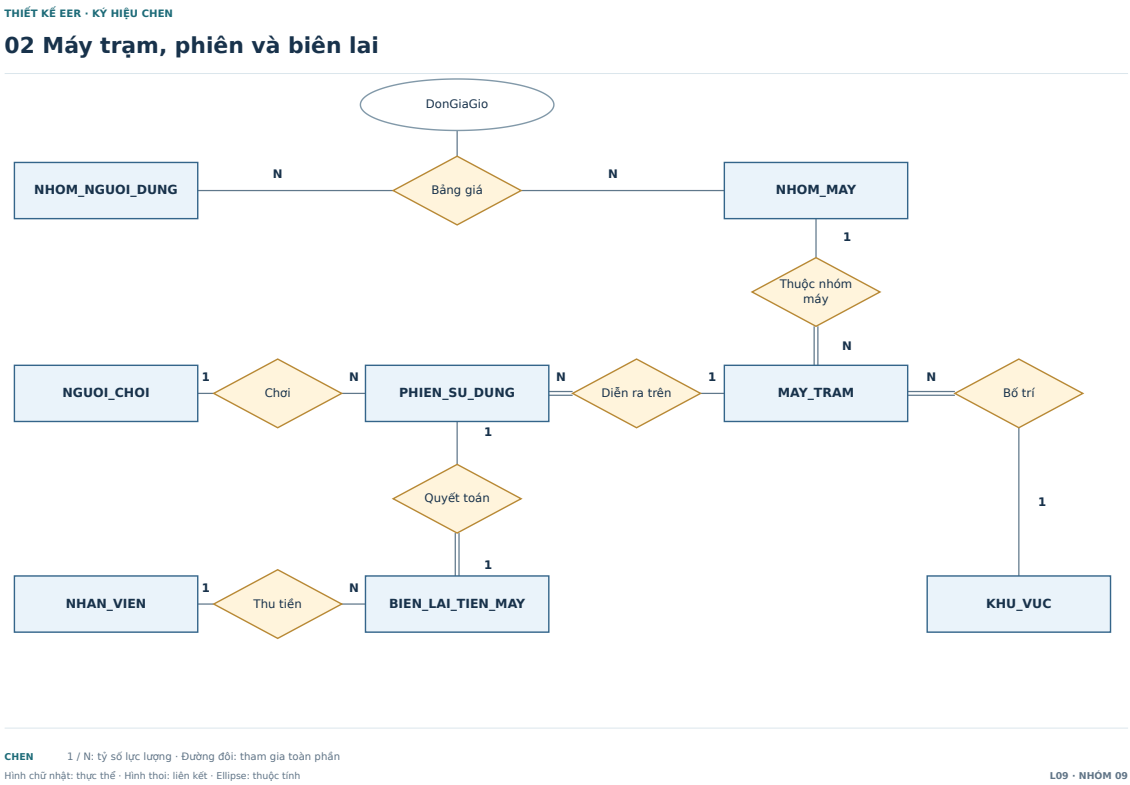
Hình B.1. EER tổng thể kế thừa từ thiết kế BTL1

B.2 Danh tính và người dùng



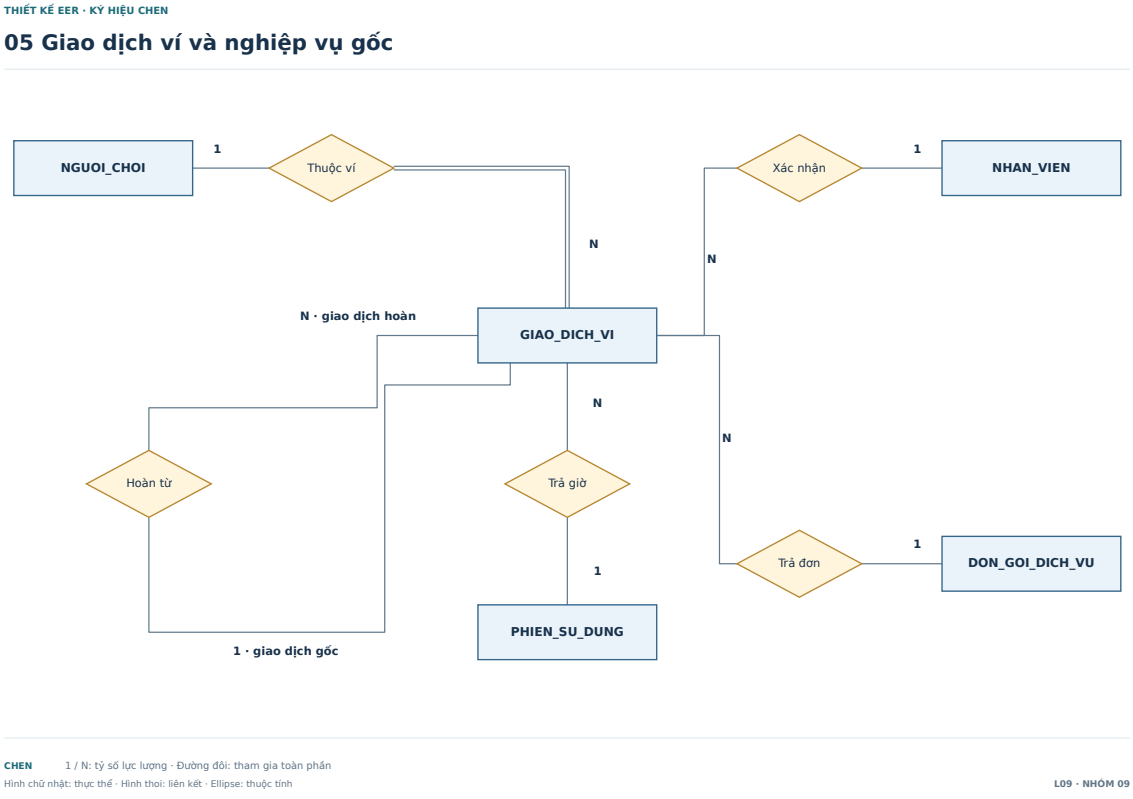
Hình B.2. EER phân hệ danh tính và người dùng

B.3 Máy trạm và phiên sử dụng



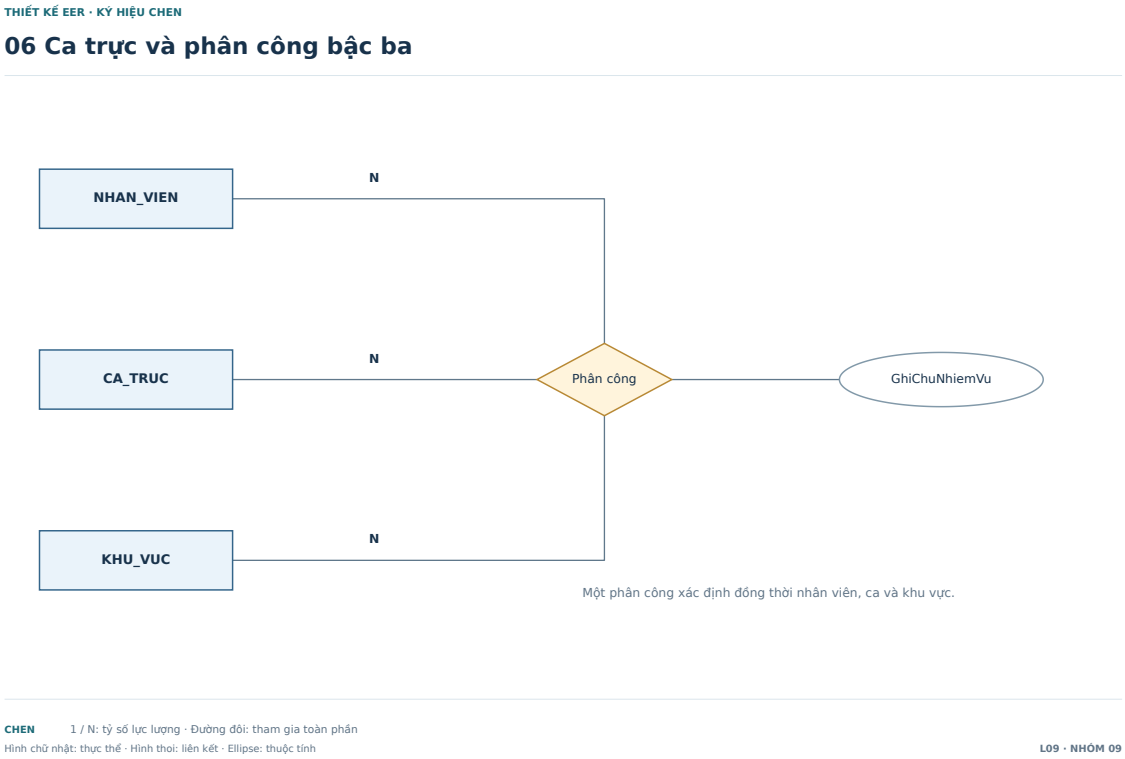
Hình B.3. EER phân hệ máy trạm và phiên sử dụng

B.4 Ví hội viên và chứng từ



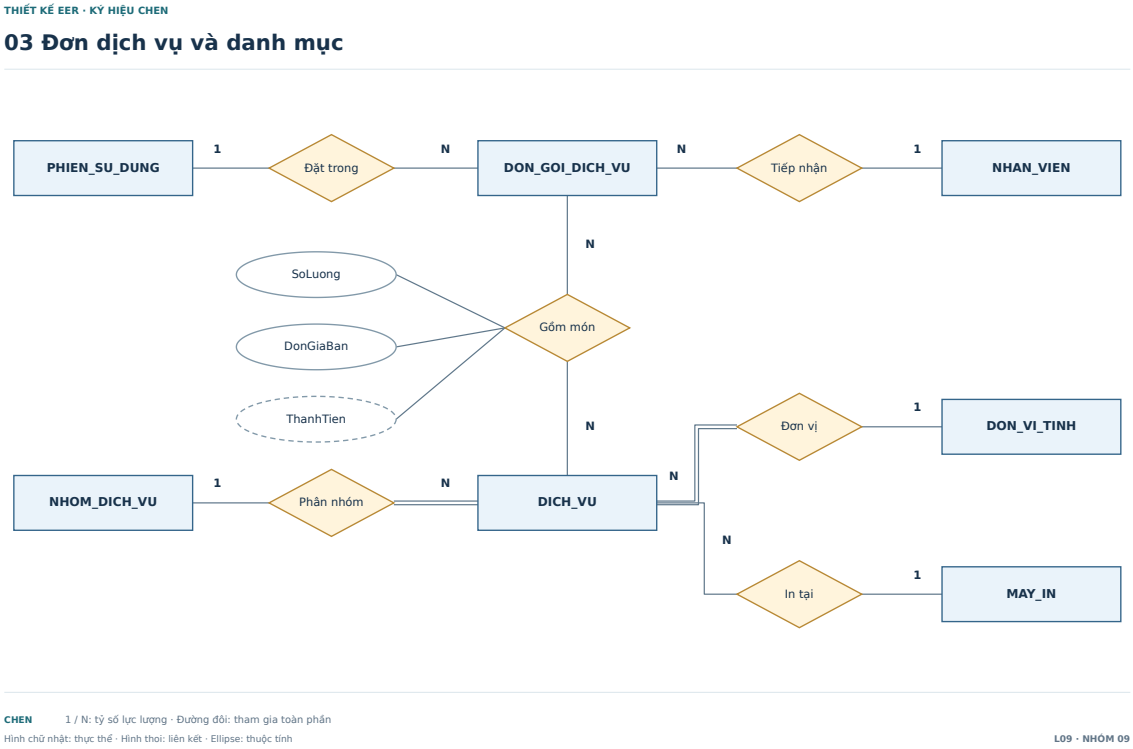
Hình B.4. EER phân hệ ví hội viên và chứng từ

B.5 Ca trực và phân công



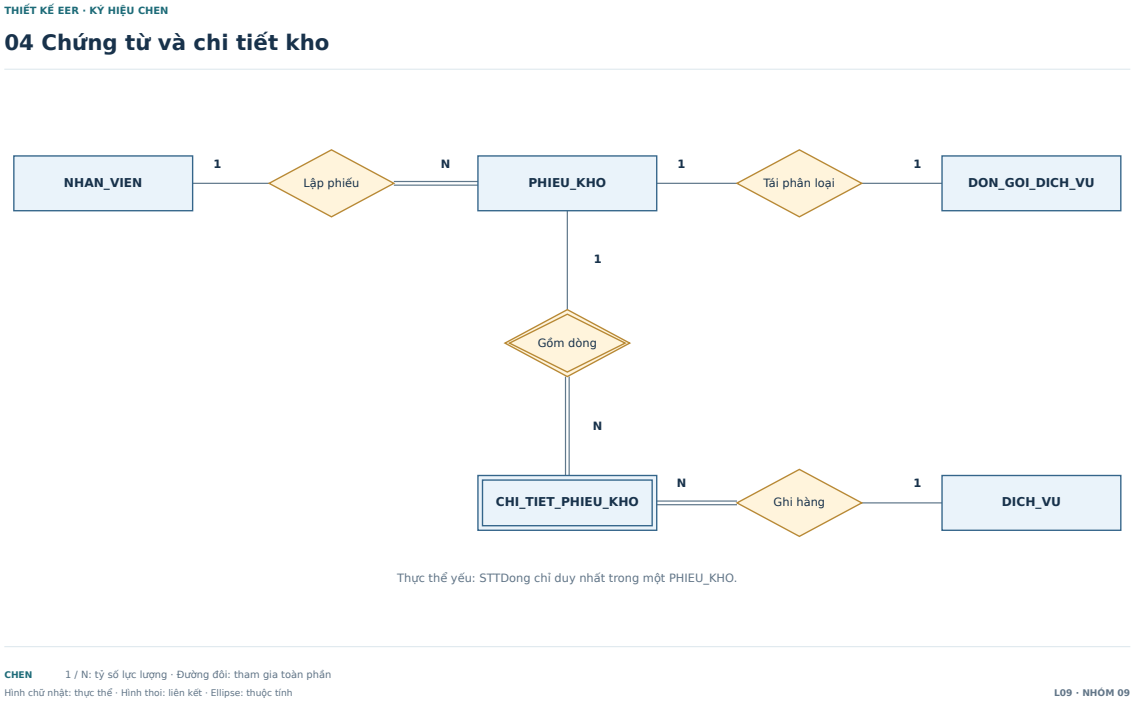
Hình B.5. EER phân hệ ca trực và phân công

B.6 Dịch vụ và đơn hàng



Hình B.6. EER phân hệ dịch vụ và đơn hàng

B.7 Phiếu kho và kiểm kê



Hình B.7. EER phân hệ phiếu kho và kiểm kê

B.8 Bảng ghi chú các thay đổi so với BTL1

Nội dung thay đổi	Phiên bản cũ (BTL1)	Phiên bản mới (BTL2)	Lí do
Duy nhất có điều kiện	Chỉ có ràng buộc quan hệ thực thể, chưa ràng buộc mức DBMS chống phiên mở đồng thời.	Bổ sung cột sinh và chỉ mục UNIQUE có điều kiện cho phiên đang hoạt động.	Ngăn chặn triệt để 1 tài khoản mở nhiều phiên hoặc 1 máy trạm mở cùng lúc 2 phiên.
Đồng bộ tổng tiền đơn	Tổng tiền đơn tính độc lập hoặc dựa vào logic tầng ứng dụng.	Bộ 3 AFTER trigger trên chi tiết dịch vụ tự động đồng bộ lại tổng tiền đầu đơn.	Bảo toàn tính nhất quán dữ liệu kế toán, không phụ thuộc ứng dụng ngoài.
Phân định ngày doanh thu	Thống kê doanh thu theo ngày tạo đơn hoặc ngày tạo phiên.	Tách riêng thời điểm thanh toán biên lai/đơn và thời điểm hoàn tiền.	Phản ánh đúng dòng tiền thực tế theo ngày phát sinh thay vì ngày đặt đơn.
Miền giá trị tiền tệ trong hàm	Dùng INT hoặc DECIMAL ngắn trong các biến tính toán của hàm.	Chuẩn hóa sang DECIMAL(18,0) cho toàn bộ biến tổng và kết quả trả về của hàm.	Tránh tràn số và sai lệch số học khi tổng tiền hoặc giao dịch ví quy mô lớn.
Miền chuỗi Cursor trong thủ tục	Khai báo biến tạm VARCHAR(50) ngắn trong thân thủ tục.	Khai báo khớp độ dài lược đồ (VARCHAR(100) cho tên, VARCHAR(30) cho đơn vị).	Tránh lỗi cắt cụt chuỗi dữ liệu (Data truncation) khi duyệt báo cáo tồn kho.
Tối ưu hóa chỉ mục thực đơn	Chỉ có các khóa chính và khóa ngoại mặc định theo thiết kế ERD.	Tạo chỉ mục phức hợp `ix_dich_vu_filter` trên `(trang_thai, ma_nhom, ten_dich_vu)`.	Giảm chi phí đọc và tăng tốc truy vấn lọc thực đơn 3.88x (đo bằng EXPLAIN ANALYZE).

PHẦN C

Dữ liệu mẫu và nguồn minh chứng

Mỗi hình dưới là bản xuất của SQL client trong cùng phiên kiểm chứng với các truy vấn ở Phần 1. Các dữ liệu là dữ liệu demo; không sử dụng thông tin cá nhân thật. Phần SQL và kết quả thô đầy đủ đi kèm ở thư mục evidence/dbms.

MySQL SQL Client

Máy chủ MySQL 8.4.13 · Kết nối local · Phiên kiểm chứng BTL2 L09 Nhóm 10

q01_data

```
SELECT MaMay,MaNhomMay,MaKhuVuc,TrangThaiMay FROM MAY_TRAM;
SELECT * FROM KHU_VUC;
SELECT MaNhomMay,TenNhomMay,DonGiaGioMacDinh FROM NHOM_MAY;
```

Thực thi SQL btl2_evidence_a62ff5712250 · Hoàn tất

6 dòng · 2.17 ms

MaMay	MaNhomMay	MaKhuVuc	TrangThaiMay
M01	VIP	KV01	AVAILABLE
M02	VIP	KV01	AVAILABLE
M03	VIP	KV01	MAINTENANCE
M04	VIP	KV01	AVAILABLE
M05	VIP	KV02	ONLINE
M06	VIP	KV02	OFFLINE

2 dòng · 5.04 ms

MaKhuVuc	TenKhuVuc	MoTaViTri
KV01	Khu thường	NULL
KV02	Khu khác	NULL

1 dòng · 2.80 ms

MaNhomMay	TenNhomMay	DonGiaGioMacDinh
VIP	Máy VIP	15000

SQL được gửi tới MySQL thật. Mỗi ảnh có câu lệnh và đầu ra JSON tương ứng trong evidence/dbms.

Hình C.1. Các bảng dữ liệu mẫu liên quan Q-P12-01

MySQL SQL Client

Máy chủ MySQL 8.4.13 · Kết nối local · Phiên kiểm chứng BTL2 L09 Nhóm 10

q02_data

```
SELECT MaDichVu,TenDichVu,GiaBanHienHanh,MaNhomDV,MaDonVi FROM DICH_VU;
SELECT * FROM NHOM_DICH_VU;
SELECT * FROM DON_VI_TINH;
```

Thực thi SQL btl2_evidence_a62ff5712250 · Hoàn tất

6 dòng · 3.13 ms

MaDichVu	TenDichVu	GiaBanHienHanh	MaNhomDV	MaDonVi
DV01	Mì trứng	15000	NDV01	DVT01
DV02	Nước ngọt	12000	NDV02	DVT02
DV03	Cơm chiên trứng	25000	NDV01	DVT01
DV04	Bò húc lon	18000	NDV02	DVT02
DV05	Cà phê sữa	15000	NDV02	DVT02
DV06	Trà chanh sả	10000	NDV02	DVT02

2 dòng · 2.34 ms

MaNhomDV	TenNhomDV
NDV01	Đồ ăn
NDV02	Đồ uống

2 dòng · 2.66 ms

MaDonVi	TenDonVi
DVT02	Chai
DVT01	Phần

SQL được gửi tới MySQL thật. Mỗi ảnh có câu lệnh và đầu ra JSON tương ứng trong evidence/dbms.

Hình C.2. Các bảng dữ liệu mẫu liên quan Q-P12-02

MySQL SQL Client

Máy chủ MySQL 8.4.13 · Kết nối local · Phiên kiểm chứng BTL2 L09 Nhóm 10

q03_data

```
SELECT MaDonHang,ThoiDiemDat,KenhDat,TongTien,TrangThaiThanhToan FROM DON_GOI_DICH_VU;
```

Thực thi SQL btl2_evidence_a62ff5712250 · Hoàn tất

2 dòng · 3.62 ms

MaDonHang	ThoiDiemDat	KenhDat	TongTien	TrangThaiThanhToan
DH_P12_01	2026-10-20 11:00:00	TaiQuay	52000	DaThanhToan
DH_P12_02	2026-10-21 15:00:00	TaiQuay	50000	DaThanhToan

SQL được gửi tới MySQL thật. Mỗi ảnh có câu lệnh và đầu ra JSON tương ứng trong evidence/dbms.

Hình C.3. Các bảng dữ liệu mẫu liên quan Q-P12-03

MySQL SQL Client

Máy chủ MySQL 8.4.13 · Kết nối local · Phiên kiểm chứng BTL2 L09 Nhóm 10

q04_data

```
SELECT MaNhanVien,MaCa,MaKhuVuc FROM PHAN_CONG_TRUC;
SELECT MaCa,ThoiDiemBatDau,ThoiDiemKetThuc FROM CA_TRUC;
SELECT MaTaiKhoan,ChucVu FROM NHAN_VIEN;
```

Thực thi SQL btl2_evidence_a62ff5712250 · Hoàn tất

5 dòng · 2.16 ms

MaNhanVien	MaCa	MaKhuVuc
TK01	CA02	KV01
TK03	CA01	KV01
TK07	CA02	KV01
TK07	CA03	KV01
TK03	CA01	KV02

3 dòng · 4.63 ms

MaCa	ThoiDiemBatDau	ThoiDiemKetThuc
CA01	2026-10-26 07:00:00	2026-10-26 12:00:00
CA02	2026-10-26 12:00:00	2026-10-26 17:00:00
CA03	2026-10-26 17:00:00	2026-10-26 22:00:00

4 dòng · 2.23 ms

MaTaiKhoan	ChucVu
TK01	QuanLy
TK03	ThuNgan
TK06	ThuNgan
TK07	PhaChe

SQL được gửi tới MySQL thật. Mỗi ảnh có câu lệnh và đầu ra JSON tương ứng trong evidence/dbms.

Hình C.4. Các bảng dữ liệu mẫu liên quan Q-P12-04

MySQL SQL Client
Máy chủ MySQL 8.4.13 · Kết nối local · Phiên kiểm chứng BTL2 L09 Nhóm 10

q05_data

```
SELECT MaTaiKhoan,SoDuVi FROM NGUOI_CHOI;
SELECT MaNguoiChoi,LoaiGiaoDich,SoTien,TrangThaiGD FROM GIAO_DICH_VI;
```

Thực thi SQL btl2_evidence_a62ff5712250 · Hoàn tất

7 dòng · 2.68 ms

MaTaiKhoan	SoDuVi
TK02	100000
TK04	100000
TK05	100000
TK06	100000
TK08	100000
TK09	250000
TK10	50000

3 dòng · 3.20 ms

MaNguoiChoi	LoaiGiaoDich	SoTien	TrangThaiGD
TK02	NapTien	50000	ThanhCong
TK02	TruTienMay	15000	ThanhCong
TK09	NapTien	200000	ThanhCong

SQL được gửi tới MySQL thật. Mỗi ảnh có câu lệnh và đầu ra JSON tương ứng trong evidence/dbms.

Hình C.5. Các bảng dữ liệu mẫu liên quan Q-P12-05

C.1 Nguồn và khả năng tái tạo

- Đề gốc và mẫu Updated: thư mục `docs/requirements/` và tệp mẫu trong `docs/` của gói bàn giao.
- DDL, khóa và routine: `database/01_schema/` đến `database/08_triggers/`.
- Minh chứng trực tiếp: `evidence/dbms/manifest.json`; mỗi mục ánh xạ ảnh, câu SQL và kết quả JSON.
- Số đo chỉ mục: `evidence/queries/index_benchmark.json`; kiểm thử: `evidence/final_verification/`.
- Nguồn sơ đồ EER: `docs/btl1_baseline/BTL1_EER.drawio`; các bản vector dùng trong báo cáo nằm tại `assets/eer/` của bộ LaTeX.
- Nguồn báo cáo: `main.tex`, các tệp `sections/`, `diagrams/` và `sql/`. Biên dịch bằng XeLaTeX để giữ nguyên tiếng Việt và phông chữ.